

Cappella Website

Animation Systems Report — Full Code

Every animation on the site: where it is used, how it works, and its complete, unabridged source code.

Source file: Cappella Website.dc.html (3268 lines, single-file Figma replication)
All code blocks below are verbatim extracts with original line numbers noted.
Generated: 14 July 2026 · Branch: main

Overview

All animation code lives in `Cappella Website.dc.html`. CSS animations sit in the `<style>` block (lines 46–448); JavaScript-driven animations are implemented as `_setup*()` methods on the page class, booted from `componentDidMount()` (line 490). The site combines three engines:

Engine	Used for
Plain CSS transitions / keyframes	Scroll reveals, menu overlay, film grain, hover states, underline wipes
GSAP + ScrollTrigger + Lenis	Smooth scrolling, scroll-scrubbed parallax, curtain reveal, S-curve draw, split-text
Raw canvas / WebGL render loops	Water-flow dot grid, Bayer dithers, GLSL hover shader, logo marquee

Contents

- 01

Preloader logo sequence
- 02

Hero intro: grid lines + character wave
- 03

Hero headline masks + scroll-out
- 04

Water-flow dot grid
- 05

Text scramble (nav + portfolio)
- 06

Generic scroll reveals
- 07

Hero parallax zoom
- 08

Line-split paragraph reveals
- 09

Odometer stat counters
- 10

Whitespace staircase reveal
- 11

Journey: S-curve draw + phase slider
- 12

Dither-resolve photo entries
- 13

Portfolio WebGL dither hover + parallax
- 14

“Cappella Edge” accordion
- 15

Footer: curtain reveal + staged rise + live clock
- 16

Operators logo marquee
- 17

Menu overlay
- 18

Film grain
- 19

Inactive systems
- 20

Animation wiring + cleanup

01 — Preloader logo sequence

`_setupPreloader()` · lines 1254-1386

CSS keyframes · lines 127-137

Where: Fires immediately on page load, covering the whole viewport.

How: A fixed red curtain with a centered logo/wordmark. The logo pops in at 200 ms, the name slides up at 400 ms; at 1100 ms both images measure the real header logo via `getBoundingClientRect()` and fly to the navbar (FLIP-style) while the red curtain slides off to the top-left, then everything is removed.

Full function — `_setupPreloader()` (lines 1254-1386)

```
_setupPreloader() {
  // --- OCI-style sequential loader ---
  document.body.style.overflow = 'hidden';

  // Full-screen red background
  const preloader = document.createElement('div');
  preloader.style.cssText =
    'position:fixed; inset:0; background:rgb(209,32,47); z-index:9999;' +
    'transition:transform 1.0s cubic-bezier(0.77,0,0.175,1);';
  document.body.appendChild(preloader);

  // Hide the actual header logo on page initially so we don't see double
  const hideHeaderWordmark = () => {
    const el = document.querySelector('.fig-asset-7cb777f5a65019d1-0d894d80');
    if (el) {
      el.style.opacity = '0';
      el.style.transition = 'opacity 0.25s ease';

      // Add logo icon to header and shift wordmark
      if (!document.getElementById('cap-header-logo-icon')) {
        const logoIcon = document.createElement('div');
        logoIcon.id = 'cap-header-logo-icon';
        logoIcon.style.cssText = 'position:absolute; left:32px; top:44px; width:34px; height:37px;
background:url(./components/assets/eae7f9bdb03982f5.png) center/contain no-repeat; opacity:0;
filter:brightness(0) invert(1); z-index:90;';
        el.parentNode.appendChild(logoIcon);
        el.style.left = '76px';
      }
    } else {
      setTimeout(hideHeaderWordmark, 50);
    }
  };
  hideHeaderWordmark();

  // --- Center logo content ---
  const centerWrap = document.createElement('div');
  centerWrap.style.cssText =
    'position:fixed; inset:0; z-index:10000; pointer-events:none;' +
    'display:flex; flex-direction:column; align-items:center; justify-content:center;';
  document.body.appendChild(centerWrap);

  const logoImg = document.createElement('img');
  logoImg.src = './components/assets/eae7f9bdb03982f5.png';
  logoImg.style.cssText =
    'width:90px; height:auto; opacity:0; transform:scale(0.7);' +
    'transition:transform 0.6s cubic-bezier(0.16,1,0.3,1), opacity 0.5s ease;' +
    'filter:brightness(0) invert(1);'; // Make it white on red
  centerWrap.appendChild(logoImg);

  const nameImg = document.createElement('img');
  nameImg.src = './components/assets/7cb777f5a65019d1.png';
  nameImg.style.cssText =
    'width:160px; height:auto; margin-top:18px; opacity:0;' +
    'transform:translateY(12px);' +
```

```

    'transition:transform 0.6s cubic-bezier(0.16,1,0.3,1), opacity 0.5s ease;' +
    'filter:brightness(0) invert(1);' // White on red
    centerWrap.appendChild(nameImg);

// === Sequential timing (no crosshair lines, no circles) ===
// Step 1: Logo icon pops in (200ms)
setTimeout(() => {
    logoImg.style.opacity = '1';
    logoImg.style.transform = 'scale(1)';
}, 200);

// Step 2: Name text slides up (400ms)
setTimeout(() => {
    nameImg.style.opacity = '1';
    nameImg.style.transform = 'translateY(0)';
}, 400);

// Step 3: Exit - fly to navbar header wordmark while fading out logo and merging name (1100ms)
setTimeout(() => {
    document.body.style.overflow = '';

    const vp = document.getElementById('cap-viewport');
    const s = vp ? vp.clientWidth / 1440 : 1;
    const hw = window.innerWidth / 2;
    const hh = window.innerHeight / 2;

    const headerWordmark = document.querySelector('.fig-asset-7cb777f5a65019d1-0d894d80');
    const logoIcon = document.getElementById('cap-header-logo-icon');

    let nameTx = 0, nameTy = 0, nameScale = 1;
    let logoTx = 0, logoTy = 0, logoScale = 1;

    if (headerWordmark && logoIcon) {
        const nameRect = nameImg.getBoundingClientRect();
        const wordmarkRect = headerWordmark.getBoundingClientRect();
        nameTx = wordmarkRect.left - nameRect.left;
        nameTy = wordmarkRect.top - nameRect.top;
        nameScale = wordmarkRect.width / nameRect.width;

        const logoRect = logoImg.getBoundingClientRect();
        const logoIconRect = logoIcon.getBoundingClientRect();
        logoTx = logoIconRect.left - logoRect.left;
        logoTy = logoIconRect.top - logoRect.top;
        logoScale = logoIconRect.height / logoRect.height;
    } else {
        // Fallback to absolute scale and coords if not loaded
        const targetX = 76 * s;
        const targetY = 44 * s;
        nameTx = -hw + targetX + 50 * s;
        nameTy = -hh + targetY + 16 * s;
        nameScale = 0.48 * s;

        const logoTargetX = 32 * s;
        logoTx = -hw + logoTargetX + 20 * s;
        logoTy = -hh + targetY + 16 * s;
        logoScale = 0.15 * s;
    }

    // CAPPELLA name flies toward navbar header wordmark
    nameImg.style.transition =
        'transform 0.85s cubic-bezier(0.77,0,0.175,1), opacity 0.65s ease';
    nameImg.style.transform = `translate(${nameTx}px, ${nameTy}px) scale(${nameScale})`;

    // Logo flies next to the wordmark and merges cleanly
    logoImg.style.transition =
        'transform 0.85s cubic-bezier(0.77,0,0.175,1), opacity 0.85s ease';
    logoImg.style.transform = `translate(${logoTx}px, ${logoTy}px) scale(${logoScale})`;
    logoImg.style.opacity = '1';

    // Red curtain slides off to top-left corner simultaneously

```

```

preloader.style.transform = 'translate(-100%, -100%)';

// Clean up and restore header logo opacity when preloader completes
setTimeout(() => {
  if (headerWordmark) headerWordmark.style.opacity = '1';
  if (logoIcon) logoIcon.style.opacity = '1';
  centerWrap.remove();
  preloader.remove();
}, 950);
}, 1100);
}

```

CSS — logo stroke-draw keyframes (currently unused by the sequential loader) (lines 127-137)

```

/* Preloader Logo drawing animation */
.cap-logo-path {
  stroke-dasharray: 600;
  stroke-dashoffset: 600;
  animation: capDraw 0.8s cubic-bezier(0.25, 1, 0.5, 1) forwards;
}
@keyframes capDraw {
  to {
    stroke-dashoffset: 0;
  }
}

```

02 — Hero intro: grid lines + character wave

`_setupHeroIntro()` · lines 925-998

Where: Hero section, right after the preloader exits.

How: Vertical hairline SVGs scale in from `scaleX(0)`, staggered 80 ms apart; every large hero text is split into per-character spans inside overflow-hidden word wrappers, each char rising from `translateY(110%)` with a 20 ms-per-char cascade.

Full function — `_setupHeroIntro()` (lines 925-998)

```
_setupHeroIntro() {
  const scaler = document.getElementById('cap-scaler');
  if (!scaler) return;

  // 1. VERTICAL GRID LINES REVEAL (scales scaleX from 0 to 1 for rotated 90deg SVGs)
  const svgs = [...scaler.querySelectorAll('svg')].filter(el => {
    const top = parseFloat(el.style.top) || 0;
    if (top > 1000) return false;
    const transform = el.style.transform || '';
    return transform.includes('matrix(0,1,-1,0') || transform.includes('matrix(0, 1, -1, 0)');
  });

  svgs.forEach((el, idx) => {
    const originalTransform = el.style.transform || '';
    el.style.transformOrigin = '0 0';
    el.style.transform = originalTransform + ' scaleX(0)';
    el.style.transition = 'transform 1.6s cubic-bezier(0.16, 1, 0.3, 1)';

    setTimeout(() => {
      el.style.transform = originalTransform + ' scaleX(1)';
    }, 100 + idx * 80);
  });

  // 2. SPLIT-TEXT CHAR STAGGER WAVE REVEAL
  const headings = [...scaler.querySelectorAll('span, p, h1, h2, h3')].filter(el => {
    const top = parseFloat(el.style.top) || 0;
    if (top > 850) return false;
    const style = window.getComputedStyle(el);
    const fontSize = parseFloat(style.fontSize) || 0;
    const txt = (el.textContent || '').trim();
    return fontSize >= 24 && txt.length > 3 && !el.querySelector('*') && txt !== 'CAPPELLA';
  });

  headings.forEach((el) => {
    const originalText = el.textContent || '';
    el.innerHTML = '';

    const words = originalText.split(/\s+/);
    const wordSpans = words.map(word => {
      const charSpans = word.split('').map(char => {
        const s = document.createElement('span');
        s.textContent = char;
        s.style.display = 'inline-block';
        s.style.opacity = '0';
        s.style.transform = 'translateY(110%)';
        s.style.transition = 'transform 0.8s cubic-bezier(0.215, 0.61, 0.355, 1), opacity 0.8s ease';
        return s;
      });
    });

    const wordContainer = document.createElement('span');
    wordContainer.style.display = 'inline-block';
    wordContainer.style.overflow = 'hidden';
    wordContainer.style.whiteSpace = 'nowrap';
    wordContainer.style.verticalAlign = 'bottom';
```

```
wordContainer.style.marginRight = '0.22em';

charSpans.forEach(cs => wordContainer.appendChild(cs));
return { container: wordContainer, chars: charSpans };
});

wordSpans.forEach(ws => el.appendChild(ws.container));

const allChars = wordSpans.flatMap(ws => ws.chars);

setTimeout(() => {
  allChars.forEach((cs, cIdx) => {
    setTimeout(() => {
      cs.style.opacity = '1';
      cs.style.transform = 'translateY(0)';
    }, cIdx * 20);
  });
}, 250);
});
}
```

03 — Hero headline masks + scroll-out

`_setupHeroTitle()` · lines 1000-1090

Where: The 100px headline “India’s Leading / Edu-Infra Asset / Management Company”.

How: Rebuilds the `<br>`-separated span into per-line overflow masks; characters enter from `translateY(115%) rotate(6deg)` timed to the preloader exit; a GSAP `ScrollTrigger` scrub then drifts each line up and fades it on scroll — lower lines faster, for a parallax exit.

Full function — `_setupHeroTitle()` (lines 1000-1090)

```
_setupHeroTitle() {
  // The 100px headline ("India's Leading / Edu-Infra Asset / Management
  // Company") is a single span with <br> children, so the generic char
  // reveal above skips it (!el.querySelector('*')). Animate it here:
  // per-line masks, chars rising with a settle rotation, then a
  // scroll-linked exit where each line drifts up into its mask and fades.
  const scaler = document.getElementById('cap-scaler');
  const heroTitle = scaler && [...scaler.querySelectorAll('span')].find(el => {
    const top = parseFloat(el.style.top) || 0;
    const fs = parseFloat(window.getComputedStyle(el).fontSize) || 0;
    return top < 850 && fs >= 90;
  });
  if (!heroTitle) {
    if ((this._htTries = (this._htTries || 0) + 1) < 40) setTimeout(() => this._setupHeroTitle(), 200);
    return;
  }
  if (heroTitle.dataset.capAnimated) return;
  heroTitle.dataset.capAnimated = '1';
  if (window.matchMedia('(prefers-reduced-motion: reduce)').matches) return;

  // Rebuild the span preserving its <br> line structure
  const lines = [];
  let current = [];
  [...heroTitle.childNodes].forEach(n => {
    if (n.nodeType === Node.ELEMENT_NODE && n.tagName === 'BR') {
      lines.push(current.join(' '));
      current = [];
    } else if (n.textContent && n.textContent.trim()) {
      current.push(n.textContent.trim());
    }
  });
  if (current.length) lines.push(current.join(' '));
  if (!lines.length) return;

  heroTitle.innerHTML = '';
  const lineEls = lines.map(lineText => {
    const lineMask = document.createElement('span');
    lineMask.style.cssText = 'display:block; overflow:hidden; white-space:nowrap;';
    const lineInner = document.createElement('span');
    lineInner.style.cssText = 'display:block; will-change:transform;';
    const chars = [];
    lineText.split(/\\s+/).forEach((word, wi, arr) => {
      const w = document.createElement('span');
      w.style.cssText = 'display:inline-block; white-space:nowrap;' +
        (wi < arr.length - 1 ? ' margin-right:0.25em;' : '');
      word.split('').forEach(ch => {
        const c = document.createElement('span');
        c.textContent = ch;
        c.style.cssText =
          'display:inline-block; opacity:0;' +
          'transform:translateY(115%) rotate(6deg); transform-origin:0% 100%;' +
          'transition: transform 1.05s cubic-bezier(0.19, 1, 0.22, 1), opacity 0.5s ease;';
        w.appendChild(c);
        chars.push(c);
      });
    });
    lineInner.appendChild(w);
    lineMask.appendChild(lineInner);
    heroTitle.appendChild(lineMask);
  });
}
```

```

    });
    lineInner.appendChild(w);
  });
  lineMask.appendChild(lineInner);
  heroTitle.appendChild(lineMask);
  return { lineInner, chars };
});

// Staggered entrance: line by line, chars cascading within each line.
// Base delay syncs with the preloader wipe (slides away at ~1.1s) so the
// reveal is actually visible instead of finishing behind the red screen.
lineEls.forEach((L, li) => {
  L.chars.forEach((c, ci) => {
    setTimeout(() => {
      c.style.opacity = '1';
      c.style.transform = 'translateY(0) rotate(0deg)';
    }, 1150 + li * 140 + ci * 20);
  });
});

// Scroll-linked parallax exit - lower lines leave faster for depth
if (window.gsap && window.ScrollTrigger) {
  lineEls.forEach((L, li) => {
    window.gsap.to(L.lineInner, {
      y: -(40 + li * 30),
      opacity: 0,
      ease: 'none',
      scrollTrigger: {
        trigger: document.getElementById('cap-viewport'),
        start: 'top top',
        end: () => '+' + window.innerHeight * 0.8,
        scrub: true
      }
    });
  });
}
}

```


04 — Water-flow dot grid

`_setupDotGrid()` · lines 718-890

canvas markup · line 455

Where: Canvas overlaying the hero image.

How: A ~130,000-dot grid rendered through a single ImageData pixel buffer per rAF frame with a sine lookup table. Each dot eases toward a target displacement: ambient sine shimmer + smoothstep cursor bulge/drag + gaussian ripple rings shed by fast strokes and clicks. Idles while the hero is off-screen.

Markup — the canvas element (lines 455-455)

```
<canvas id="cap-dotgrid" width="1440" height="830" style="position: absolute; left: 0; top: 0; width: 1440px; height: 830px; z-index: 2; pointer-events: none;"></canvas>
```

Full function — `_setupDotGrid()` (lines 718-890)

```
_setupDotGrid() {
  const canvas = document.getElementById('cap-dotgrid');
  if (!canvas) { if ((this._dgTries = (this._dgTries || 0) + 1) < 40) setTimeout(() =>
this._setupDotGrid(), 200); return; }
  const ctx = canvas.getContext('2d');
  const W = 1440, H = 830, dotSize = 1.4, gap = 1.6, cell = dotSize + gap;
  const baseA = 0.16, activeA = 0.95;
  const cols = Math.floor((W + gap) / cell), rows = Math.floor((H + gap) / cell);
  const gridW = cell * cols - gap, gridH = cell * rows - gap;
  const sx = (W - gridW) / 2 + dotSize / 2, sy = (H - gridH) / 2 + dotSize / 2;
  const dots = [];
  for (let y = 0; y < rows; y++) for (let x = 0; x < cols; x++) {
    const cx = sx + x * cell, cy = sy + y * cell;
    dots.push({ cx, cy, ox: 0, oy: 0,
      p1: cx * 0.011 + cy * 0.006, p2: cy * 0.018, p3: cx * 0.009 + cy * 0.004 });
  }
  // Pixel-buffer renderer: 130k+ fillRect calls per frame run at slideshow
  // speed; writing pixels into one ImageData and blitting it once is ~60fps.
  canvas.width = W; canvas.height = H;
  const frame = ctx.createImageData(W, H);
  const px32 = new Uint32Array(frame.data.buffer);
  // sine lookup table — 3 trig calls x 130k dots per frame is the other hotspot
  const SIN_N = 4096, SIN_MASK = SIN_N - 1, SIN_K = SIN_N / (Math.PI * 2);
  const SIN = new Float32Array(SIN_N);
  for (let i = 0; i < SIN_N; i++) SIN[i] = Math.sin((i / SIN_N) * Math.PI * 2);
  const fsin = (v) => SIN[((v * SIN_K) | 0) & SIN_MASK];
  const fcos = (v) => SIN[((v * SIN_K) | 0) + SIN_N / 4 & SIN_MASK];

  // ===== WATER-FLOW CURSOR FIELD =====
  // No impulse kicks: every frame each dot eases toward a continuous target
  // displacement (surface bulge + directional drag + ripple rings), which is
  // what makes the response read as fluid instead of twitchy.
  const P = { x: -9999, y: -9999, lastX: 0, lastY: 0, lastT: 0, vx: 0, vy: 0 };
  let smoothX = -9999, smoothY = -9999;
  let sVX = 0, sVY = 0; // smoothed cursor velocity (the wake direction)
  const R = 150, R2 = R * R; // cursor influence radius
  const BULGE = 16; // how far the surface parts around the cursor
  const rings = []; // expanding click/wake ripples
  let lastWake = 0;
  let isMoving = false;
  let cursorGlow = 0;
  let dgIdleTimeout = null;

  const sc = () => { const e = document.getElementById('cap-scaler'); const m = (e.style.transform || '').match(/scale\(([^\)]+)\)/); return m ? parseFloat(m[1]) : 1; };
  const onMove = (e) => {
    const s = sc(), r = canvas.getBoundingClientRect(), now = performance.now(), dt = P.lastT ?
Math.max(now - P.lastT, 1) : 16;
```

```

const nx = (e.clientX - r.left) / s, ny = (e.clientY - r.top) / s;
P.vx = (nx - P.lastX) / dt * 1000; P.vy = (ny - P.lastY) / dt * 1000;
P.lastX = nx; P.lastY = ny; P.lastT = now; P.x = nx; P.y = ny;

isMoving = true;
clearTimeout(dgIdleTimeout);
dgIdleTimeout = setTimeout(() => {
  isMoving = false;
}, 250); // 250ms idle threshold

// fast strokes shed small wake ripples behind the cursor, like a hand through water
const speed = Math.hypot(P.vx, P.vy);
if (speed > 550 && now - lastWake > 110 && ny > -50 && ny < H + 50) {
  lastWake = now;
  rings.push({ x: nx, y: ny, r: 10, speed: 3.2, amp: 7, sigma: 22 });
}
};
const onClick = (e) => {
  const s = sc(), r = canvas.getBoundingClientRect();
  const cx = (e.clientX - r.left) / s, cy = (e.clientY - r.top) / s;
  if (cy < -50 || cy > H + 50) return;
  // a dropped-stone ripple: main crest + a softer follower
  rings.push({ x: cx, y: cy, r: 6, speed: 5.2, amp: 22, sigma: 30 });
  rings.push({ x: cx, y: cy, r: -34, speed: 5.2, amp: 12, sigma: 24 });
};
window.addEventListener('mousemove', onMove, { passive: true });
window.addEventListener('mouseleave', () => { P.x = -9999; P.y = -9999; isMoving = false; });
window.addEventListener('click', onClick);

const draw = () => {
  // Hero off-screen -> skip all physics/painting, just idle the loop
  const rect = canvas.getBoundingClientRect();
  if (rect.bottom < 0 || rect.top > window.innerHeight) {
    this._dgRaf = requestAnimationFrame(draw);
    return;
  }
  const T = (this._dgT = (this._dgT || 0) + 0.025); // ambient water time
  px32.fill(0);

  // Spring-smoothed cursor position (organic tracking)
  if (P.x > -9000) {
    if (smoothX < -9000) { smoothX = P.x; smoothY = P.y; this._dgVX = 0; this._dgVY = 0; }
    if (!this._dgVX) { this._dgVX = 0; this._dgVY = 0; }
    this._dgVX += (P.x - smoothX) * 0.07; this._dgVY += (P.y - smoothY) * 0.07;
    this._dgVX *= 0.78; this._dgVY *= 0.78;
    smoothX += this._dgVX; smoothY += this._dgVY;
  } else { smoothX = -9999; smoothY = -9999; }

  // Smoothly update cursorGlow multiplier (fast fade in, slow decay on idle)
  const targetGlow = (isMoving && P.x > -9000) ? 1.0 : 0.0;
  cursorGlow += (targetGlow - cursorGlow) * (isMoving ? 0.12 : 0.06);
  canvas.style.opacity = cursorGlow;

  // Smoothed velocity - decays gently so the wake trails and settles
  sVX += (((P.x > -9000) ? P.vx : 0) - sVX) * 0.06;
  sVY += (((P.x > -9000) ? P.vy : 0) - sVY) * 0.06;
  const dragX = Math.max(-30, Math.min(30, sVX * 0.022));
  const dragY = Math.max(-30, Math.min(30, sVY * 0.022));

  // Advance ripples
  for (let i = rings.length - 1; i >= 0; i--) {
    const g = rings[i];
    g.r += g.speed;
    g.amp *= 0.962;
    if (g.amp < 0.35 || g.r - 4 * g.sigma > Math.hypot(W, H)) rings.splice(i, 1);
  }

  for (const d of dots) {
    // ambient flowing-water shimmer
    const wave = fsin(d.p1 + T) + fsin(d.p2 - T * 1.25);

```

```

const wx = fcos(d.p3 + T * 0.9) * 2.6;
const wy = wave * 2.4;
let a = baseA * (0.45 + 0.55 * (wave * 0.5 + 0.5));

// target displacement this frame
let tx = 0, ty = 0;

// 1. surface bulge + directional drag around the cursor (only active/visible when cursorGlow > 0)
const dx = d.cx - smoothX, dy = d.cy - smoothY, dsq = dx * dx + dy * dy;
const currentR2 = R2 * cursorGlow;
if (cursorGlow > 0.005 && dsq <= currentR2) {
  const dist = Math.sqrt(dsq) || 0.5;
  const currentR = R * Math.sqrt(cursorGlow);
  let t = 1 - dist / currentR;
  t = t * t * (3 - 2 * t); // smoothstep falloff
  tx += (dx / dist) * BULGE * t * cursorGlow + dragX * t * cursorGlow;
  ty += (dy / dist) * BULGE * t * cursorGlow + dragY * t * cursorGlow;
  a = Math.max(a, baseA + (activeA - baseA) * t * cursorGlow);
}

// 2. ripple rings - a gaussian crest passing through the dot
for (const g of rings) {
  if (g.r < 0) continue;
  const bb = g.r + 3.5 * g.sigma;
  const gx = d.cx - g.x, gy = d.cy - g.y;
  if (gx > bb || gx < -bb || gy > bb || gy < -bb) continue;
  const gd = Math.sqrt(gx * gx + gy * gy) || 0.5;
  const off = gd - g.r;
  if (off > 3.5 * g.sigma || off < -3.5 * g.sigma) continue;
  const crest = Math.exp(-(off * off) / (2 * g.sigma * g.sigma));
  const push = g.amp * crest;
  tx += (gx / gd) * push;
  ty += (gy / gd) * push;
  a = Math.min(0.95, a + crest * (g.amp / 22) * 0.5); // the crest glints
}

// ease toward the target - this is the water smoothness
d.ox += (tx - d.ox) * 0.09;
d.oy += (ty - d.oy) * 0.09;

const ix = (d.cx + d.ox + wx) | 0, iy = (d.cy + d.oy + wy) | 0;
if (ix >= 0 && ix < W && iy >= 0 && iy < H) {
  const A = (a * 255) | 0;
  px32[iy * W + ix] = (A << 24) | 0x00FFFFFF; // white, alpha A (little-endian)
  if (a > 0.45 && ix + 1 < W && iy + 1 < H) {
    // bright dots (cursor glow / ripple crests) get a 2x2 glint
    const A2 = (a * 140) | 0;
    const half = (A2 << 24) | 0x00FFFFFF;
    px32[iy * W + ix + 1] = half;
    px32[(iy + 1) * W + ix] = half;
  }
}
}
ctx.putImageData(frame, 0, 0);
this._dgRaf = requestAnimationFrame(draw);
};
draw();
}

```

05 — Text scramble (nav + portfolio)

`_setupNavScramble()` · lines 892–923

`wrapInCharSpans/scrambleText` · lines 2652–2789

Where: Header nav links; city filters and school names in the Portfolio section.

How: On hover, a 30–32 ms interval rewrites each character with random glyphs, locking in the real character left-to-right as a frame counter passes its index. The portfolio version wraps chars in spans and re-aligns the red C logo every frame.

Full function — `_setupNavScramble()` (lines 892–923)

```
_setupNavScramble() {
  const labels = ['ABOUT US', 'PROJECTS', 'TEAM', 'CONTACT US'];
  const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ#%*<>/'
  const spans = [...document.querySelectorAll('#cap-scaler span')].filter((s) => {
    const t = (s.textContent || '').trim();
    const r = s.getBoundingClientRect();
    return labels.includes(t) && r.top < 300;
  });
  if (spans.length < 4 && (this._navTries = (this._navTries || 0) + 1) < 40) {
    setTimeout(() => this._setupNavScramble(), 200);
    return;
  }
  spans.forEach((el) => {
    const target = (el.textContent || '').trim();
    el.classList.add('cap-nav'); // nowrap exception — keeps CONTACT US on one line
    el.style.cursor = 'pointer';
    let iv = null;
    el.addEventListener('mouseenter', () => {
      let frame = 0;
      clearInterval(iv);
      iv = setInterval(() => {
        el.textContent = target.split('').map((c, i) => {
          if (c === ' ') return ' ';
          if (i < frame / 2.4) return target[i];
          return chars[Math.floor(Math.random() * chars.length)];
        }).join('');
        frame++;
        if (frame / 2.4 >= target.length) { clearInterval(iv); el.textContent = target; }
      }, 32);
    });
  });
}
```

Portfolio version — `wrapInCharSpans()`, `alignLogo()`, `scrambleText()` and bindings (lines 2652–2789)

```
const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ#%*<>/'

// Wrap target string in individual span tags for character-by-character layout tracking
const wrapInCharSpans = (el) => {
  // innerText keeps rendered line breaks (nested nodes/<br>) as \n, which
  // textContent drops — rebuilding from textContent glued words together
  const originalText = (el.innerText || el.textContent || '').replace(/\u2028/g, '\n');
  el.innerHTML = '';
  const spans = originalText.split('').map(c => {
    const s = document.createElement('span');
    s.textContent = c;
    s.className = 'cap-char';
    // Keep chars inline: inline-block spans collapse whitespace to zero
    // width and allow line breaks between letters mid-word
    if (/\/s/.test(c)) s.style.whiteSpace = 'pre-wrap';
    el.appendChild(s);
    return s;
  });
}
```

```

    return spans;
  };

  // Keep the red interlocking C logo perfectly positioned to the right of the active city name
  // ROOT CAUSE FIX: hide the logo immediately so it never shows at its default Figma
  // position (left:309, top:4464) which overlaps the 'Hyderabad' text. Only show it
  // after alignLogo() has successfully re-positioned it.
  (() => {
    const logoEl = scaler.querySelector('.fig-asset-eae7f9bdb03982f5-132fb376');
    if (logoEl) {
      logoEl.style.opacity = '0';
      logoEl.style.transition = 'opacity 0.3s ease';
    }
  })();

  let fontsRegistered = false;
  const alignLogo = () => {
    const logoEl = scaler.querySelector('.fig-asset-eae7f9bdb03982f5-132fb376');
    if (!logoEl) return;

    const activeCityEl = cityEls.find(c => c.name === currentCityName)?.el;
    if (!activeCityEl) return;

    // Filter out trailing spaces, line breaks, zero-width chars - only count actual letters
    const charSpans = [...activeCityEl.querySelectorAll('.cap-char')].filter(span => {
      return /[a-zA-Z]/.test(span.textContent || '');
    });

    if (charSpans && charSpans.length > 0) {
      const lastChar = charSpans[charSpans.length - 1];
      const lastCharRect = lastChar.getBoundingClientRect();
      const parentRect = (logoEl.offsetParent || scaler).getBoundingClientRect();
      const s = sc();

      // getBoundingClientRect can return 0-rect for off-screen elements on some browsers.
      // Fall back to reading the span's inline style (Figma coordinates) in that case.
      if (lastCharRect.width === 0 && lastCharRect.height === 0) {
        // Use the city el's own style top/left as a fallback absolute position
        const cityTop = parseFloat(activeCityEl.style.top) || 0;
        const cityLeft = parseFloat(activeCityEl.style.left) || 0;
        const cityWidth = parseFloat(activeCityEl.style.width) || 120;
        const cityHeight = parseFloat(activeCityEl.style.height) || 28;
        logoEl.style.left = (cityLeft + cityWidth + 12) + 'px';
        logoEl.style.top = (cityTop + (cityHeight - 19) / 2) + 'px';
      } else {
        // Position the logo (21px wide, 19px tall) 12px to the right of the last character
        logoEl.style.left = ((lastCharRect.right - parentRect.left) / s + 12) + 'px';
        // Vertically center the logo with the last character
        logoEl.style.top = ((lastCharRect.top - parentRect.top) / s + (lastCharRect.height - 19) / 2) +
'px';
      }

      // Reveal the logo now that it's correctly positioned
      logoEl.style.opacity = '1';
    }

    if (!fontsRegistered) {
      fontsRegistered = true;
      const runAlign = () => {
        alignLogo();
        requestAnimationFrame(alignLogo);
      };
      if (document.fonts) {
        document.fonts.ready.then(runAlign);
      }
      setTimeout(runAlign, 100);
      setTimeout(runAlign, 350);
      setTimeout(runAlign, 800);
      setTimeout(runAlign, 1500);
    }
  }

```

```

});

const scrambleText = (el, originalText) => {
  // Keep indices aligned with wrapInCharSpans' innerText-based content
  originalText = (el.innerText || originalText).replace(/\u2028/g, '\n');
  const charSpans = wrapInCharSpans(el);
  let interval;
  let step = 0;

  el.addEventListener('mouseenter', () => {
    clearInterval(interval);
    interval = setInterval(() => {
      charSpans.forEach((cs, index) => {
        const char = originalText[index];
        if (char === ' ' || char === '\n' || char === '\u2028') return;
        if (index < step) {
          cs.textContent = char;
        } else {
          cs.textContent = chars[Math.floor(Math.random() * chars.length)];
        }
      });
      alignLogo(); // Align during scramble frames
      step += 1;
      if (step > originalText.length) {
        clearInterval(interval);
        charSpans.forEach((cs, index) => {
          cs.textContent = originalText[index];
        });
        alignLogo();
        step = 0;
      }
    }, 30);
  });

  el.addEventListener('mouseleave', () => {
    clearInterval(interval);
    charSpans.forEach((cs, index) => {
      cs.textContent = originalText[index];
    });
    alignLogo();
    step = 0;
  });
});

cityEls.forEach(c => {
  scrambleText(c.el, c.el.textContent || '');
});

cards.forEach(c => {
  scrambleText(c.label, c.label.textContent || '');
});

```

06 — Generic scroll reveals

CSS · lines 46-106

_setupScrollReveals() · lines 1092-1252

Where: Section headings, stat numbers, and body paragraphs across the whole page.

How: JS classifies elements heuristically and tags them with reveal classes; an IntersectionObserver adds .cap-revealed once at 12% visibility. All motion is pure CSS, gated behind prefers-reduced-motion.

CSS — reveal base classes (lines 46-106)

```
/* ===== SCROLL ANIMATION BASE STYLES ===== */
@media (prefers-reduced-motion: no-preference) {
  /* Elements that will be revealed on scroll */
  .cap-scroll-reveal {
    opacity: 0;
    transform: translateY(40px);
    transition: opacity 0.75s cubic-bezier(0.22, 1, 0.36, 1),
      transform 0.75s cubic-bezier(0.22, 1, 0.36, 1);
    will-change: opacity, transform;
  }
  .cap-scroll-reveal.cap-revealed {
    opacity: 1;
    transform: translateY(0);
  }
  /* Stagger delays for child items */
  .cap-scroll-reveal.cap-delay-1 { transition-delay: 0.1s; }
  .cap-scroll-reveal.cap-delay-2 { transition-delay: 0.2s; }
  .cap-scroll-reveal.cap-delay-3 { transition-delay: 0.3s; }
  .cap-scroll-reveal.cap-delay-4 { transition-delay: 0.4s; }
  .cap-scroll-reveal.cap-delay-5 { transition-delay: 0.5s; }

  /* Slide-in from left */
  .cap-slide-left {
    opacity: 0;
    transform: translateX(-60px);
    transition: opacity 0.8s cubic-bezier(0.22, 1, 0.36, 1),
      transform 0.8s cubic-bezier(0.22, 1, 0.36, 1);
    will-change: opacity, transform;
  }
  .cap-slide-left.cap-revealed {
    opacity: 1;
    transform: translateX(0);
  }

  /* Scale-up reveal */
  .cap-scale-reveal {
    opacity: 0;
    transform: scale(0.88);
    transition: opacity 0.7s cubic-bezier(0.22, 1, 0.36, 1),
      transform 0.7s cubic-bezier(0.22, 1, 0.36, 1);
    will-change: opacity, transform;
  }
  .cap-scale-reveal.cap-revealed {
    opacity: 1;
    transform: scale(1);
  }

  /* Clip reveal — text slides up from a masked box */
  .cap-clip-reveal {
    overflow: hidden;
  }
  .cap-clip-reveal > * {
    display: block;
    transform: translateY(100%);
    transition: transform 0.85s cubic-bezier(0.22, 1, 0.36, 1);
  }
}
```

```

    will-change: transform;
  }
  .cap-clip-reveal.cap-revealed > * {
    transform: translateY(0);
  }
}

```

Full function — `_setupScrollReveals()` (also boots `Lenis`, nav links, footer email) (lines 1092–1252)

```

_setupScrollReveals() {
  if (window.matchMedia('(prefers-reduced-motion: reduce)').matches) return;
  const scaler = document.getElementById('cap-scaler');
  if (!scaler) return;

  // Helper: get element's true top in page (px), accounting for scaler scale
  const getScaledTop = (el) => {
    const s = parseFloat((scaler.style.transform || 'scale(1)').match(/scale\(((^)+)\)/)?.[1] || '1');
    const r = el.getBoundingClientRect();
    return r.top + window.scrollY;
  };

  // --- Classify elements by which section they're likely in ---
  // Strategy: find key heading/block elements by text content and position,
  // then add reveal classes + observe with IntersectionObserver.

  const addReveal = (el, cls = 'cap-scroll-reveal', delay = '') => {
    el.classList.add(cls);
    if (delay) el.classList.add(delay);
  };

  // Observe and fire
  const io = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        entry.target.classList.add('cap-revealed');
        io.unobserve(entry.target);
      }
    });
  });

  // SECTION HEADINGS — find large text nodes that are section titles
  const sectionHeadings = [...scaler.querySelectorAll('span, div')].filter(el => {
    const txt = (el.textContent || '').trim();
    const style = window.getComputedStyle(el);
    const fontSize = parseFloat(style.fontSize) || 0;
    return fontSize >= 40 && txt.length > 3 && txt.length < 60;
  });
  sectionHeadings.slice(0, 20).forEach((el, i) => {
    // Skip elements in the hero (visible on page load)
    const r = el.getBoundingClientRect();
    if (r.top < window.innerHeight) return;
    addReveal(el, 'cap-scroll-reveal');
    io.observe(el);
  });

  // STAT NUMBERS — large metric text ($500 Mn+, 135 Acres, 16 Assets, 75 Years+)
  const statNums = [...scaler.querySelectorAll('span, div')].filter(el => {
    const txt = (el.textContent || '').trim();
    return /^(?!\d+\s*\w*[+])$/.test(txt) || txt.match(/^\d.*[+)$/);
  });
  statNums.forEach((el, i) => {
    const r = el.getBoundingClientRect();
    if (r.top < window.innerHeight) return;
    addReveal(el, 'cap-scale-reveal', 'cap-delay-' + Math.min(i + 1, 5));
    io.observe(el);
  });

  // BODY PARAGRAPHS — description text blocks
  const bodyParas = [...scaler.querySelectorAll('p, span')].filter(el => {

```



```

    const txt = (el.textContent || '').trim();
    const style = window.getComputedStyle(el);
    const fontSize = parseFloat(style.fontSize) || 0;
    return fontSize >= 16 && fontSize < 36 && txt.length > 40;
  });
  bodyParas.forEach((el, i) => {
    const r = el.getBoundingClientRect();
    if (r.top < window.innerHeight) return;
    addReveal(el, 'cap-scroll-reveal', 'cap-delay-' + (i % 3 + 1));
    io.observe(el);
  });

  this._scrollRevealIO = io;

  // Initialize Lenis Smooth Scroll & Sync GSAP Ticker
  if (window.Lenis && window.gsap && window.ScrollTrigger) {
    this._lenis = new window.Lenis({
      duration: 1.2,
      easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)), // OCI style easing
    });
    this._lenis.on('scroll', window.ScrollTrigger.update);
    window.gsap.ticker.add((time) => {
      this._lenis.raf(time * 1000);
    });
    window.gsap.ticker.lagSmoothing(0);
  }

  this._setupHeroZoom();
  this._setupSplitText();

  // ===== MENU SMOOTH SCROLLING =====
  const navItems = [
    { text: 'ABOUT US', target: 900 },
    { text: 'PROJECTS', target: 3000 },
    { text: 'PORTFOLIO', target: 3000 },
    { text: 'TEAM', target: 6200 },
    { text: 'CONTACT US', target: 6942 }
  ];

  setTimeout(() => {
    navItems.forEach(item => {
      const els = [...scaler.querySelectorAll('span')].filter(el =>
        (el.textContent || '').replace(/\s+/g, '').toUpperCase() === item.text.replace(/\s+/g, ''));
      els.forEach(el => {
        el.style.cursor = 'pointer';
        el.addEventListener('click', () => {
          const vp = document.getElementById('cap-viewport');
          const target = item.target * ((vp ? vp.clientWidth : window.innerWidth) / 1440);
          if (this._lenis) {
            this._lenis.scrollTo(target);
          } else {
            window.scrollTo({ top: target, behavior: 'smooth' });
          }
        });
      });
    });
  }, 500);

  // ===== FOOTER EMAIL INPUT =====
  setTimeout(() => {
    const emailSpan = [...scaler.querySelectorAll('span')].find(el =>
      (el.textContent || '').trim() === 'Your Email Id');
    if (emailSpan) {
      const input = document.createElement('input');
      input.type = 'email';
      input.placeholder = 'Your Email Id';
      input.style.position = 'absolute';
      input.style.left = emailSpan.style.left || '1124px';
    }
  });

```

```

input.style.top = emailSpan.style.top || '345px';
input.style.width = emailSpan.style.width || '180px';
input.style.height = '35px';
input.style.background = 'transparent';
input.style.border = 'none';
input.style.outline = 'none';
input.style.fontFamily = emailSpan.style.fontFamily || 'Montserrat, sans-serif';
input.style.fontSize = emailSpan.style.fontSize || '14px';
input.style.fontWeight = emailSpan.style.fontWeight || '500';
input.style.color = 'rgb(255,255,255)';
input.style.borderBottom = '1px solid rgba(255,255,255,0.4)';

emailSpan.parentNode.replaceChild(input, emailSpan);

const arrowSvg = [...input.parentNode.querySelectorAll('svg')].find(s =>
  s.style.left === '1307px' || (parseFloat(s.style.left) >= 1300 && parseFloat(s.style.left) <=
1320)
);
if (arrowSvg) {
  arrowSvg.style.cursor = 'pointer';
  arrowSvg.addEventListener('click', () => {
    if (input.value) {
      alert(`Thank you for subscribing, ${input.value}!`);
      input.value = '';
    } else {
      alert('Please enter a valid email address.');
```

07 — Hero parallax zoom

`_setupHeroZoom()` · lines 1388-1412

Where: The hero photo wrapper.

How: GSAP scrubs the hero block to `scale(1.15)` over the first 1200 px of scroll, anchored at bottom-center.

Full function — `_setupHeroZoom()` (lines 1388-1412)

```
_setupHeroZoom() {
  // 4. Parallax Hero Zoom
  if (!window.gsap || !window.ScrollTrigger) return;
  const scaler = document.getElementById('cap-scaler');
  if (!scaler) return;

  const heroWrapper = [...scaler.querySelectorAll('div')].find(e => {
    const style = e.getAttribute('style') || '';
    return style.includes('width: 1440') && style.includes('height: 830.204');
  });

  if (heroWrapper) {
    heroWrapper.style.transformOrigin = '50% 100%'; // Zoom from bottom center slightly
    window.gsap.to(heroWrapper, {
      scale: 1.15,
      ease: "none",
      scrollTrigger: {
        trigger: document.body,
        start: "top top",
        end: "1200px", // Scroll distance
        scrub: true
      }
    });
  }
}
```

08 — Line-split paragraph reveals

`_setupSplitText()` · lines 1414–1464

Where: All long body paragraphs.

How: `SplitType` splits each paragraph into rendered lines; every line is wrapped in an overflow-hidden mask at `yPercent:100` and GSAP staggers them up on first entry.

Full function — `_setupSplitText()` (lines 1414–1464)

```
_setupSplitText() {
  // 5. Staggered Line-Split Quote Reveal
  if (!window.gsap || !window.ScrollTrigger || !window.SplitType) return;
  const scaler = document.getElementById('cap-scaler');
  if (!scaler) return;

  // Apply to main body paragraphs (the ones used for cap-scroll-reveal)
  const bodyParas = [...scaler.querySelectorAll('p, span')].filter(el => {
    const txt = (el.textContent || '').trim();
    const style = window.getComputedStyle(el);
    const fontSize = parseFloat(style.fontSize) || 0;
    return fontSize >= 16 && fontSize < 36 && txt.length > 40;
  });

  bodyParas.forEach((el) => {
    // Clean up previous classes to avoid conflicts
    el.classList.remove('cap-scroll-reveal');
    el.style.opacity = '1';
    el.style.transform = 'none';

    // Use SplitType to split text into lines
    const split = new window.SplitType(el, { types: 'lines' });

    // Wrap each line in an overflow hidden mask
    split.lines.forEach(line => {
      const wrapper = document.createElement('div');
      wrapper.style.overflow = 'hidden';
      wrapper.style.display = 'inline-block';
      wrapper.style.verticalAlign = 'top';
      wrapper.style.padding = '0';
      wrapper.style.margin = '0';
      line.parentNode.insertBefore(wrapper, line);
      wrapper.appendChild(line);
      // Initial state for GSAP
      window.gsap.set(line, { yPercent: 100 });
    });

    window.ScrollTrigger.create({
      trigger: el,
      start: "top 90%",
      onEnter: () => {
        window.gsap.to(split.lines, {
          yPercent: 0,
          duration: 1.2,
          ease: "power4.out",
          stagger: 0.08
        });
      }
    });
  });
}
```

09 — Odometer stat counters

`_setupStatCounters()` · lines 2196-2259

Where: Every large stat figure — \$500 Mn+, 135 Acres, 16 Assets, 75 Years+, 10 Years.

How: Each digit becomes a masked 1em window over a column of 0–9 twice; on first visibility the column snaps to 0 and rolls to $-(10+\text{digit})\text{em}$ so every digit passes one full cycle before landing, staggered 90 ms per digit.

Full function — `_setupStatCounters()` (lines 2196-2259)

```
_setupStatCounters() {
  // PREMIUM PASS: odometer digit rolls. Every large stat figure (leaf nodes,
  // >=55px — catches "$500", "135 Acres", "16 Assets", "75 Years+", "10 Years")
  // rolls each digit through a masked 0-9 column into place.
  const scaler = document.getElementById('cap-scaler');
  if (!scaler) return;
  const reduced = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
  const els = [...scaler.querySelectorAll('div, span')].filter(el => {
    if (el.querySelector('*') || el.dataset.capOdo) return false;
    const txt = (el.textContent || '').trim();
    if (!txt || txt.length > 12 || !/\d/.test(txt)) return false;
    const fs = parseFloat(window.getComputedStyle(el).fontSize) || 0;
    return fs >= 55;
  });
  els.forEach(el => {
    el.dataset.capOdo = '1';
    if (reduced) return; // leave static text
    const txt = (el.textContent || '').trim();
    el.textContent = '';
    const rolls = [];
    [...txt].forEach(ch => {
      if (!/\d/.test(ch)) {
        const s = document.createElement('span');
        s.textContent = ch;
        s.style.whiteSpace = 'pre';
        el.appendChild(s);
        return;
      }
      // Mask holds a column of 0-9 twice; rolling to 10+digit passes a full
      // cycle before landing. Initially shows the real digit (offset digit).
      const digit = parseInt(ch, 10);
      const mask = document.createElement('span');
      mask.style.cssText = 'display:inline-block; overflow:hidden; height:1em; line-height:1em; vertical-align:baseline;';
      const col = document.createElement('span');
      col.style.cssText = 'display:block; will-change:transform; transform:translateY(-' + digit + 'em)';
      for (let r = 0; r < 2; r++) {
        for (let d = 0; d <= 9; d++) {
          const cell = document.createElement('span');
          cell.textContent = String(d);
          cell.style.cssText = 'display:block; height:1em; line-height:1em;';
          col.appendChild(cell);
        }
      }
      mask.appendChild(col);
      el.appendChild(mask);
      rolls.push({ col, digit });
    });
    if (!rolls.length) return;
    const io = new IntersectionObserver((entries) => {
      entries.forEach(entry => {
        if (!entry.isIntersecting) return;
        io.disconnect();
        rolls.forEach(({ col, digit }, i) => {
```

```
col.style.transition = 'none';
col.style.transform = 'translateY(0)';
void col.offsetHeight; // reflow so the jump to 0 doesn't animate
col.style.transition = 'transform 1.5s var(--cap-ease-expo) ' + (i * 0.09) + 's';
col.style.transform = 'translateY(-' + (10 + digit) + 'em)';
});
});
}, { threshold: 0.4 });
io.observe(e1);
});
}
```

10 — Whitespace staircase reveal

markup · lines 457–475

_setupReveal() · lines 2281–2321

Where: The white staircase notch cutting into the red stats band, under “135 Acres”.

How: Four white bars (25%→100% wide) pre-collapsed with `scaleX(0)` inside a clipped container; scroll position maps to a 0–1 progress and each bar gets a staggered slice of it (bottom first), eased with `easeInOutQuad`. Scrubs backwards on scroll-up. The “75 Years+” stat fades in during the tail.

Markup — staircase bars + red band fill + stat (lines 457–475)

```
<!-- Fill the white staircase notch in the red band with red (keeps band solid) -->
<div style="position: absolute; left: 0; top: 1715px; width: 1440px; height: 440px; background:
rgb(209,32,47); z-index: 4;"></div>

<!-- 75 Years+ stat on solid red (no staircase behind it) -->
<div id="cap-75" style="position: absolute; left: 695px; top: 1360px; z-index: 5;">
  <div style="font-family: Montserrat, sans-serif; font-weight: 600; font-size: 60px; line-height: 1;
color: rgb(255,255,255);">75 Years+</div>
  <div style="font-family: Montserrat, sans-serif; font-weight: 400; font-size: 24px; line-height: 1;
color: rgb(255,255,255); margin-top: 16px;">Leadership Experience</div>
</div>

<!-- New glowing point (matches the 16 Assets point) after 135 Acres -->
<div id="cap-dot2" style="position: absolute; left: 102px; top: 1950px; width: 88px; height: 88px; z-
index: 5; border-radius: 50%; background: radial-gradient(44px 44px, rgb(255,255,255) 0%, rgb(209,32,47)
55%);"></div>

<!-- Scroll-linked staircase reveal — left half only, starting under "135 Acres" -->
<div id="cap-reveal" style="position: absolute; left: 0px; top: 1715px; width: 728px; height: 440px; z-
index: 6; pointer-events: none; overflow: hidden;">
  <div class="cap-stair" style="position: absolute; left: 0; top: 0px; width: 25%; height: 110px;
background: rgb(255,255,255); transform-origin: left; transform: scaleX(0); will-change: transform;"></div>
  <div class="cap-stair" style="position: absolute; left: 0; top: 110px; width: 50%; height: 110px;
background: rgb(255,255,255); transform-origin: left; transform: scaleX(0); will-change: transform;"></div>
  <div class="cap-stair" style="position: absolute; left: 0; top: 220px; width: 75%; height: 110px;
background: rgb(255,255,255); transform-origin: left; transform: scaleX(0); will-change: transform;"></div>
  <div class="cap-stair" style="position: absolute; left: 0; top: 330px; width: 100%; height: 110px;
background: rgb(255,255,255); transform-origin: left; transform: scaleX(0); will-change: transform;"></div>
</div>
```

Full function — _setupReveal() (lines 2281–2321)

```
_setupReveal() {
  const reveal = document.getElementById('cap-reveal');
  const stat75 = document.getElementById('cap-75');
  if (!reveal) return;
  const stairs = [...reveal.querySelectorAll('.cap-stair')];
  const N = stairs.length;
  const clamp = (v) => v < 0 ? 0 : v > 1 ? 1 : v;
  const easeInOutQuad = (t) => t < 0.5 ? 2 * t * t : 1 - Math.pow(-2 * t + 2, 2) / 2;

  this._onRevealScroll = () => {
    const r = reveal.getBoundingClientRect();
    const vh = window.innerHeight;

    // Calculate progress p: 0 when the reveal top is near the bottom of viewport,
    // 1 when it's well above it.
    const p = clamp((vh * 0.85 - r.top) / (vh * 0.85));

    // We want a staggered reveal starting from the bottom stair (index 3) to the top stair (index 0)
    const span = 0.55;
    const stagger = 0.15;
```

```
stairs.forEach((stair, i) => {
  // Since i=0 is top, i=1 is middle, i=2 is bottom,
  // and we want bottom (i=2) to start first, middle (i=1) second, top (i=0) third:
  const delay = (N - 1 - i) * stagger;
  const stairProgress = clamp((p - delay) / span);
  const easedProgress = easeInOutQuad(stairProgress);

  stair.style.transform = 'scaleX(' + easedProgress + ')';
});

if (stat75) {
  const fp = clamp((p - 0.45) / 0.4);
  stat75.style.opacity = String(fp);
  stat75.style.transform = 'translateY(' + (30 * (1 - fp)) + 'px)';
}
};
this._onRevealScroll();
window.addEventListener('scroll', this._onRevealScroll, { passive: true });
window.addEventListener('resize', this._onRevealScroll);
}
```


11 — Journey: S-curve draw + phase slider

`_setupJourneyStory()` · lines 1920-2194

journey CSS · lines 196-259

Where: “Our Journey” section (frame top ≈ 2959–3712).

How: The S-curve draws itself via a scroll-scrubbed dasharray tween. Phase changes (Foundation → Growth → Expansion) run a skewed white gradient sweep; backdrop/building classes are swapped mid-wipe and the new photo settles with a GSAP pop. Year dots/labels are rebuilt per phase and staggered in.

CSS — journey slider (arrows, dots) (lines 196-259)

```
/* ===== DYNAMIC JOURNEY SLIDER ===== */
#cap-journey-dynamic-wrap {
  position: absolute;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  pointer-events: none;
}
.cap-journey-arrow {
  position: absolute;
  z-index: 20;
  width: 48px;
  height: 48px;
  border-radius: 50%;
  border: 1px solid rgba(255, 255, 255, 0.3);
  background: rgba(255, 255, 255, 0.08);
  backdrop-filter: blur(6px);
  -webkit-backdrop-filter: blur(6px);
  display: flex;
  align-items: center;
  justify-content: center;
  color: #fff;
  cursor: pointer;
  pointer-events: auto;
  transition: background 0.3s ease, border-color 0.3s ease, transform 0.3s cubic-bezier(0.16, 1, 0.3, 1);
}
.cap-journey-arrow:hover {
  background: #fff;
  border-color: #fff;
  color: rgb(209, 32, 47);
  transform: scale(1.1);
}
.cap-journey-arrow svg {
  width: 18px;
  height: 18px;
  fill: none;
  stroke: currentColor;
  stroke-width: 2.5;
  stroke-linecap: round;
  stroke-linejoin: round;
}
.cap-journey-dot {
  position: absolute;
  width: 20px;
  height: 20px;
  border-radius: 50%;
  background: #fff;
  border: 4px solid rgb(209, 32, 47);
  box-shadow: 0 0 12px rgba(255, 255, 255, 0.5);
  cursor: pointer;
  pointer-events: auto;
  transform: translate(-10px, -10px);
}
```

```

    transition: transform 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275), background 0.3s ease, border-color
0.3s ease;
}
.cap-journey-dot:hover {
    transform: translate(-10px, -10px) scale(1.3);
}
.cap-journey-dot.cap-dot-active {
    background: rgb(209, 32, 47);
    border-color: #fff;
    transform: translate(-10px, -10px) scale(1.35);
    box-shadow: 0 0 20px rgba(255, 255, 255, 0.8);
}

```

Full function — `_setupJourneyStory()` (lines 1920-2194)

```

_setupJourneyStory() {
    if (window.matchMedia('(prefers-reduced-motion: reduce)').matches) return;
    const scaler = document.getElementById('cap-scaler');
    if (!scaler) return;

    // --- 1. S-curve Drawing Animation (Keep it exactly as is) ---
    const svg = [...scaler.querySelectorAll('svg')].find(el => {
        const top = parseFloat(el.style.top) || 0;
        return top > 2900 && top < 3000 &&
            parseInt(el.getAttribute('height') || '0', 10) > 600 &&
            el.querySelectorAll('path').length === 1;
    });
    if (svg) {
        const path = svg.querySelector('path');
        const stroked = path && window.getComputedStyle(path).stroke !== 'none';
        if (stroked) {
            const len = path.getTotalLength();
            path.style.strokeDasharray = len;
            window.gsap.fromTo(path, { strokeDashoffset: len }, {
                strokeDashoffset: len * 2,
                ease: 'none',
                scrollTrigger: { trigger: svg, start: 'top 82%', end: 'bottom 45%', scrub: true }
            });
        } else {
            window.gsap.fromTo(svg, { clipPath: 'inset(0 0 100% 0)' }, {
                clipPath: 'inset(0 0 0% 0)',
                ease: 'none',
                scrollTrigger: { trigger: svg, start: 'top 82%', end: 'bottom 45%', scrub: true }
            });
        }
    }

    // --- 2. Dynamic Journey Slider Setup ---
    const backdrop = scaler.querySelector('.fig-asset-9943ffc227529f04-36d3d657');
    const building = scaler.querySelector('.fig-asset-531180345e372e44-3031d3f7');
    if (!backdrop || !building) {
        if ((this._journeyTries = (this._journeyTries || 0) + 1) < 40) {
            setTimeout(() => this._setupJourneyStory(), 100);
        }
        return;
    }

    // Hide original Figma nodes/spans in the journey block (top 3080 to 3700)
    [...scaler.querySelectorAll('span')].forEach(s => {
        const top = parseFloat(s.style.top) || 0;
        const txt = (s.textContent || '').trim();
        if (top > 3080 && top < 3700 && txt !== 'Our Journey') {
            s.style.display = 'none';
        }
    });
    [...scaler.querySelectorAll('div')].forEach(d => {
        const top = parseFloat(d.style.top) || 0;
        const isDot = (d.style.borderRadius || '').includes('50%') && parseFloat(d.style.width) <= 30;
        if (top > 3080 && top < 3700 && isDot) {

```

```

    d.style.display = 'none';
  }
});

// Create dynamic content wrapper and sweep transition mask
let dynamicWrap = document.getElementById('cap-journey-dynamic-wrap');
if (dynamicWrap) dynamicWrap.remove();
dynamicWrap = document.createElement('div');
dynamicWrap.id = 'cap-journey-dynamic-wrap';
scaler.appendChild(dynamicWrap);

let sweep = document.getElementById('cap-journey-sweep');
if (sweep) sweep.remove();
sweep = document.createElement('div');
sweep.id = 'cap-journey-sweep';
sweep.style.cssText = 'position:absolute; left:0; top:2959px; width:2200px; height:753px;
background:linear-gradient(to right, rgba(255,255,255,0) 0%, rgba(255,255,255,1) 15%, rgba(255,255,255,1)
85%, rgba(255,255,255,0) 100%); z-index:15; pointer-events:none; transform:skewX(-20deg)
translateX(-110%);';
scaler.appendChild(sweep);

// Setup navigation arrows
const arrowL = document.createElement('div');
arrowL.className = 'cap-journey-arrow';
arrowL.style.cssText = 'left:45px; top:3320px;';
arrowL.innerHTML = '<svg viewBox="0 0 24 24"><polyline points="15 18 9 12 15 6"></polyline></svg>';
dynamicWrap.appendChild(arrowL);

const arrowR = document.createElement('div');
arrowR.className = 'cap-journey-arrow';
arrowR.style.cssText = 'left:1350px; top:3320px;';
arrowR.innerHTML = '<svg viewBox="0 0 24 24"><polyline points="9 18 15 12 9 6"></polyline></svg>';
dynamicWrap.appendChild(arrowR);

// Dynamic phase labels container at the bottom
const phaseLabelWrap = document.createElement('div');
phaseLabelWrap.style.cssText = 'position:absolute; left:0; top:3630px; width:1440px; height:93px;';
dynamicWrap.appendChild(phaseLabelWrap);

const phaseWord = document.createElement('span');
phaseWord.style.cssText = 'position:absolute; left:1183px; top:0; font-family:Montserrat; font-
weight:400; font-size:85px; color:#fff;';
phaseWord.textContent = 'Phase';
phaseLabelWrap.appendChild(phaseWord);

const phaseTitle = document.createElement('span');
phaseTitle.style.cssText = 'position:absolute; left:637px; top:0; width:534px; font-family:Montserrat;
font-weight:800; font-size:85px; color:#fff; text-align:right; cursor:pointer; pointer-events:auto;';
phaseLabelWrap.appendChild(phaseTitle);

// Slider Data
const JOURNEY_DATA = [
  {
    phase: "Foundation",
    backdropClass: "fig-asset-9943ffc227529f04-36d3d657",
    imageClass: "fig-asset-531180345e372e44-3031d3f7",
    years: [
      { year: "2016", count: "6 Assets", desc: "Pioneered Sale and Leaseback Model", x: 693, y: 3140 },
      { year: "2017", count: "1 Asset", desc: "", x: 474, y: 3366 },
      { year: "2018", count: "1 Asset", desc: "", x: 188, y: 3551 }
    ]
  },
  {
    phase: "Growth",
    backdropClass: "fig-asset-9f9c83f5695a1af6",
    imageClass: "fig-asset-76ba2d1ec2162f53",
    years: [
      { year: "2019", count: "3 Assets", desc: "3 Assets Acquired, 1 Asset Exited, Pioneered multi-
structure revenue", x: 315, y: 3460 },
      { year: "2021", count: "2 Assets", desc: "", x: 600, y: 3220 }
    ]
  }
]

```

```

    ]
  },
  {
    phase: "Expansion",
    backdropClass: "fig-asset-63186349250e044c-069f3173",
    imageClass: "fig-asset-0f9374cd498a11f1-a670101d",
    years: [
      { year: "2023", count: "1 Asset", desc: "Portfolio successfully structured/refinanced, Captive partnership secured", x: 188, y: 3551 },
      { year: "2024", count: "1 Asset", desc: "", x: 474, y: 3366 },
      { year: "2025", count: "3 Assets", desc: "", x: 693, y: 3140 }
    ]
  }
];

let activePhase = 0;
let activeYear = "";
let isTransitioning = false;

const renderSlide = (phaseIdx, yearValOpt = "") => {
  // Clear year elements
  const oldYears = [...dynamicWrap.querySelectorAll('.cap-dyn-year-item')];
  oldYears.forEach(el => el.remove());

  const data = JOURNEY_DATA[phaseIdx];
  phaseTitle.textContent = data.phase;

  // Select active year node (defaults to first in the list)
  const yrData = data.years;
  const targetYearData = yrData.find(y => y.year === yearValOpt) || yrData[0];
  activeYear = targetYearData.year;

  const animatedElements = [];

  yrData.forEach(y => {
    // Create Dot Node on S-curve
    const dot = document.createElement('div');
    dot.className = 'cap-journey-dot cap-dyn-year-item' + (y.year === activeYear ? ' cap-dot-active' : '');

    dot.style.left = y.x + 'px';
    dot.style.top = y.y + 'px';
    dot.addEventListener('click', () => {
      if (isTransitioning) return;
      if (y.year !== activeYear) {
        activeYear = y.year;
        renderSlide(phaseIdx, y.year);
      }
    });
    dynamicWrap.appendChild(dot);

    // Compute relative positions for details
    const isLeftCurve = y.x < 400;
    const textOffsetLeft = isLeftCurve ? 28 : 27; // Shift labels right of the node
    const lx = y.x + textOffsetLeft;
    const ly = y.y - 14;

    // Year Text
    const yearTxt = document.createElement('span');
    yearTxt.className = 'cap-dyn-year-item';
    yearTxt.style.cssText = `position:absolute; left:${lx}px; top:${ly}px; font-family:Montserrat; font-weight:800; font-size:36px; color:#fff; cursor:pointer; pointer-events:auto; line-height:100%;`;
    yearTxt.textContent = y.year;
    yearTxt.addEventListener('click', () => {
      if (isTransitioning) return;
      if (y.year !== activeYear) {
        activeYear = y.year;
        renderSlide(phaseIdx, y.year);
      }
    });
    dynamicWrap.appendChild(yearTxt);
  });
};

```

```

        animatedElements.push(yearTxt);

        // Asset Count
        const countTxt = document.createElement('span');
        countTxt.className = 'cap-dyn-year-item';
        countTxt.style.cssText = `position:absolute; left:${lx}px; top:${ly + 38}px; font-family:Montserrat;
font-weight:400; font-size:24px; color:rgb(0,0,0); line-height:100%;`;
        countTxt.textContent = y.count;
        dynamicWrap.appendChild(countTxt);
        animatedElements.push(countTxt);

        // Description Text (only show if it is the active highlighted node and has description)
        if (y.year === activeYear && y.desc) {
            const descTxt = document.createElement('span');
            descTxt.className = 'cap-dyn-year-item';
            descTxt.style.cssText = `position:absolute; left:${lx}px; top:${ly + 74}px; width:280px; font-
family:Montserrat; font-weight:700; font-size:12px; color:rgb(0,0,0); line-height:140%; opacity:0;`;
            descTxt.textContent = y.desc;
            dynamicWrap.appendChild(descTxt);
            animatedElements.push(descTxt);
        }
    });

    // Animate details fade and slide-up
    window.gsap.fromTo(animatedElements,
        { opacity: 0, y: 15 },
        { opacity: (el) => el.style.color === 'rgb(0,0,0)' ? 0.95 : 1, y: 0, duration: 0.65, stagger: 0.08,
ease: 'power2.out' }
    );
};

const transitionToPhase = (targetIdx, yearValOpt = "") => {
    if (isTransitioning) return;
    isTransitioning = true;

    // 1. Start sweep diagonal wipe
    window.gsap.set(sweep, { transform: 'skewX(-20deg) translateX(-110%)' });
    window.gsap.to(sweep, {
        transform: 'skewX(-20deg) translateX(110%)',
        duration: 0.95,
        ease: 'power3.inOut'
    });

    // 2. Perform swap at mid-wipe (450ms)
    setTimeout(() => {
        // Swap Backdrop class
        backdrop.className = JOURNEY_DATA[targetIdx].backdropClass;
        // Swap Building Class
        building.className = JOURNEY_DATA[targetIdx].imageClass;

        // Render target phase content
        activePhase = targetIdx;
        renderSlide(targetIdx, yearValOpt);

        // Build image entrance slide
        window.gsap.fromTo(building, { opacity: 0.2, scale: 0.95 }, { opacity: 1, scale: 1, duration: 0.8,
ease: 'power2.out' });
    }, 450);

    // 3. Reset sweep overlay at the end
    setTimeout(() => {
        window.gsap.set(sweep, { transform: 'skewX(-20deg) translateX(-110%)' });
        isTransitioning = false;
    }, 1000);
};

const prevPhase = () => {
    const prevIdx = (activePhase - 1 + JOURNEY_DATA.length) % JOURNEY_DATA.length;
    transitionToPhase(prevIdx);
};

```

```
const nextPhase = () => {
  const nextIdx = (activePhase + 1) % JOURNEY_DATA.length;
  transitionToPhase(nextIdx);
};

// Bind event listeners to arrows
arrowL.addEventListener('click', prevPhase);
arrowR.addEventListener('click', nextPhase);
phaseTitle.addEventListener('click', nextPhase);

// Initial render
renderSlide(0);

// Also support clicking a different phase's year from other parts if needed,
// and make our dynamic wrapper animate in smoothly when scroll triggers first enter.
window.ScrollTrigger.create({
  trigger: backdrop,
  start: 'top 80%',
  onEnter: () => {
    window.gsap.fromTo('.cap-dyn-year-item', { opacity: 0, y: 20 }, { opacity: 1, y: 0, duration: 0.8,
stagger: 0.08, ease: 'expo.out' });
  }
});
}
```

12 — Dither-resolve photo entries

`_setupDitherEntries()` · lines 1692-1760

Where: The journey full-width backdrop and the journey building photo.

How: A canvas overlay painted with a red-duotone 4×4 Bayer ordered dither of the photo (downsampled to 190 px wide). It sits at full opacity until the photo scrolls in, then fades out, resolving the image to full colour.

Full function — `_setupDitherEntries()` (lines 1692-1760)

```
_setupDitherEntries() {
  // OCI-style photo entries: the image sits as a red duotone Bayer dither
  // and resolves to full colour as it scrolls into view. Applied to the
  // journey backdrop and the journey building photo (the Edge image and
  // portfolio cards already have their own dither behaviours).
  const scaler = document.getElementById('cap-scaler');
  if (!scaler || this._ditherEntriesDone) return;
  this._ditherEntriesDone = true;
  if (window.matchMedia('(prefers-reduced-motion: reduce)').matches) return;

  const TARGETS = [
    'fig-asset-9943fffc227529f04-36d3d657', // journey full-width backdrop
    'fig-asset-531180345e372e44-3031d3f7'  // journey building photo
  ];
  const bayer = [[0,12,3,15],[8,4,11,7],[2,14,1,13],[10,6,9,5]];

  TARGETS.forEach(cls => {
    const el = scaler.querySelector('.' + CSS.escape(cls));
    if (!el) return;
    const bg = window.getComputedStyle(el).backgroundImage;
    if (!bg || bg === 'none') return;

    el.style.overflow = 'hidden';
    const canvas = document.createElement('canvas');
    canvas.style.cssText =
      'position:absolute; left:0; top:0; width:100%; height:100%;' +
      'pointer-events:none; opacity:0; transition: opacity 0.9s var(--cap-ease-power) 0.2s;';
    el.appendChild(canvas);

    const tempImg = new Image();
    tempImg.crossOrigin = 'anonymous';
    tempImg.src = bg.replace(/^url\(['"]?/, '').replace(/['"]?\)$/ , '');
    tempImg.onload = () => {
      try {
        const w = 190, h = Math.round((tempImg.height / tempImg.width) * w);
        canvas.width = w; canvas.height = h;
        const ctx = canvas.getContext('2d');
        ctx.drawImage(tempImg, 0, 0, w, h);
        const imgData = ctx.getImageData(0, 0, w, h);
        const px = imgData.data;
        for (let y = 0; y < h; y++) {
          for (let x = 0; x < w; x++) {
            const i4 = (y * w + x) * 4;
            const gray = 0.299 * px[i4] + 0.587 * px[i4 + 1] + 0.114 * px[i4 + 2];
            const light = gray > ((bayer[y % 4][x % 4] + 0.5) / 16) * 255;
            px[i4] = light ? 255 : 209;
            px[i4 + 1] = light ? 235 : 32;
            px[i4 + 2] = light ? 235 : 47;
          }
        }
        ctx.putImageData(imgData, 0, 0);
      } catch (e) { canvas.remove(); return; }

      // Dithered until the photo scrolls in, then resolve to full colour
      canvas.style.transition = 'none';
    }
  });
}
```

```
canvas.style.opacity = '1';
void canvas.offsetHeight;
canvas.style.transition = 'opacity 0.9s var(--cap-ease-power) 0.2s';
const io = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (!entry.isIntersecting) return;
    io.disconnect();
    canvas.style.opacity = '0';
  });
}, { threshold: 0.3 });
io.observe(el);
});
}
```


13 — Portfolio WebGL dither hover + parallax

`_setupPortfolioInteraction()` · lines 2323-2906

Where: The four school cards in “Our Portfolio”.

How: A GLSL fragment shader per card renders a red duotone Bayer dither; hover tweens `u_dither`→0 and `u_zoom`→1.05 on rAF so the photo resolves with a slight zoom. The method also drives city filtering, a two-column scroll parallax, corner arrow buttons, and the scramble-tracking logo alignment.

Full function — `_setupPortfolioInteraction()` (includes shaders + `createWebGLDither`) (lines 2323-2906)

```
_setupPortfolioInteraction() {
  const scaler = document.getElementById('cap-scaler');
  if (!scaler) return;

  let currentCityName = 'Hyderabad';

  // Scale helper function - must be defined before alignLogo uses it
  const sc = () => {
    const e = document.getElementById('cap-scaler');
    const m = (e.style.transform || '').match(/scale\(((\[^\]]+)\))/);
    return m ? parseFloat(m[1]) : 1;
  };

  // 1. Find city elements
  const cityNames = ['Dubai', 'Andhra Pradesh', 'Bangalore', 'Chhattisgarh', 'Chennai', 'Hyderabad',
    'Pune'];
  const cityEls = [];
  cityNames.forEach(name => {
    const el = [...scaler.querySelectorAll('span')].find(e =>
      (e.textContent || '').trim().replace(/\\u2028/g, '').trim() === name.trim()
    );
    if (el) cityEls.push({ name, el });
  });

  // Aggressively hide ALL red overlays in portfolio area - check multiple top values
  const hideRedOverlays = () => {
    [...scaler.querySelectorAll('div')].forEach(e => {
      const style = e.getAttribute('style') || '';
      const top = parseFloat(e.style.top) || 0;
      const hasRedBg = style.includes('rgba(209,32,47)') || style.includes('rgba(209, 32, 47)');
      if (top >= 3800 && top <= 5200 && hasRedBg) {
        e.style.display = 'none';
      }
    });
  };
  hideRedOverlays();
  // Also run after React potentially re-renders
  setTimeout(hideRedOverlays, 500);
  setTimeout(hideRedOverlays, 1500);

  // 2. Find school card elements
  const cards = [];

  // Helper to find absolute div by class name
  const findElByClass = (cls) => scaler.querySelector('.' + cls);

  const cardData = [
    {
      id: 1,
      imgCls: 'fig-asset-3f58066ce9ef777c-b69ac725',
      labelKeywords: ['Sancta', 'Maria'],
      cities: ['Hyderabad', 'Bangalore']
    },
    {
```

```

        id: 2,
        imgCls: 'fig-asset-016a31a2601bd9e3',
        labelKeywords: ['Michael'],
        cities: ['Hyderabad', 'Chennai']
    },
    {
        id: 3,
        imgCls: 'fig-asset-0f9374cd498a11f1-a670101d',
        labelKeywords: ['Keesara'],
        cities: ['Andhra Pradesh', 'Bangalore', 'Pune']
    },
    {
        id: 4,
        imgCls: 'fig-asset-76ba2d1ec2162f53',
        labelKeywords: ['Suchitra'],
        cities: ['Andhra Pradesh', 'Chennai', 'Dubai', 'Chhattisgarh']
    }
];

// =====
// TRUE WebGL GLSL Bayer-Matrix Dither (OCI-style, Cappella Red)
// Architecture mirrors oci-website/js/dither.js but uses:
//   dark = #D1202F (Cappella red)   light = #FFF0F0 (warm white)
// uDitherAmount uniform: 1.0 = full dither / 0.0 = full colour
// On hover → tween to 0.0 + zoom 1.05x (reveal original image)
// On leave → tween to 1.0 + zoom 1.0x (restore dither)
// =====

// --- GLSL shaders ---
const VERT_SRC = `
    attribute vec2 a_pos;
    attribute vec2 a_uv;
    varying vec2 v_uv;
    void main() {
        gl_Position = vec4(a_pos, 0.0, 1.0);
        v_uv = a_uv;
    }
`;

// 4x4 Bayer threshold matrix encoded as uniforms
// u_bayer[y*4+x] = ordered-dither threshold (0-15)
const FRAG_SRC = `
    precision mediump float;
    varying vec2 v_uv;
    uniform sampler2D u_tex;
    uniform float u_dither; // 0=colour, 1=dither
    uniform float u_zoom; // 1.0 default, 1.05 hovered
    uniform float u_bayer[16]; // 4x4 Bayer matrix values

    void main() {
        /* zoom from centre */
        vec2 uv = (v_uv - 0.5) / u_zoom + 0.5;
        vec4 col = texture2D(u_tex, uv);

        /* luminance */
        float lum = dot(col.rgb, vec3(0.299, 0.587, 0.114));

        /* Bayer threshold lookup (use mod on pixel coords) */
        int bx = int(mod(floor(gl_FragCoord.x), 4.0));
        int by = int(mod(floor(gl_FragCoord.y), 4.0));
        float idx = float(by * 4 + bx);

        /* manually index - GLSL ES 1.0 needs constant-index arrays */
        /* so we use a chain of comparisons (16 branches, GPU trivial) */
        float thr = 0.0;
        if (idx < 1.0) thr = u_bayer[ 0];
        else if (idx < 2.0) thr = u_bayer[ 1];
        else if (idx < 3.0) thr = u_bayer[ 2];
        else if (idx < 4.0) thr = u_bayer[ 3];
        else if (idx < 5.0) thr = u_bayer[ 4];
    }
`;

```

```

        else if (idx < 6.0) thr = u_bayer[ 5];
        else if (idx < 7.0) thr = u_bayer[ 6];
        else if (idx < 8.0) thr = u_bayer[ 7];
        else if (idx < 9.0) thr = u_bayer[ 8];
        else if (idx < 10.0) thr = u_bayer[ 9];
        else if (idx < 11.0) thr = u_bayer[10];
        else if (idx < 12.0) thr = u_bayer[11];
        else if (idx < 13.0) thr = u_bayer[12];
        else if (idx < 14.0) thr = u_bayer[13];
        else if (idx < 15.0) thr = u_bayer[14];
        else
            thr = u_bayer[15];

    float threshold = (thr + 0.5) / 16.0;

    /* Cappella red duotone */
    vec3 dark  = vec3(0.820, 0.125, 0.184); /* #D1202F */
    vec3 light = vec3(1.000, 0.941, 0.941); /* #FFF0F0 */
    vec3 dithered = lum > threshold ? light : dark;

    /* blend between dithered and original */
    gl_FragColor = vec4(mix(col.rgb, dithered, u_dither), col.a);
}
`;

// 4x4 Bayer matrix (same as OCI)
const BAYER16 = [0,8,2,10, 12,4,14,6, 3,11,1,9, 15,7,13,5];

/**
 * Creates a WebGL dither instance on a <canvas> element.
 * Mirrors OCI's WebGLDither class: exposes showOriginal() / showDither()
 * which tween uDitherAmount + uZoom with requestAnimationFrame.
 */
function createWebGLDither(canvasEl, imgUrl) {
    const gl = canvasEl.getContext('webgl') || canvasEl.getContext('experimental-webgl');
    if (!gl) return null;

    // compile shader
    const compile = (type, src) => {
        const s = gl.createShader(type);
        gl.shaderSource(s, src);
        gl.compileShader(s);
        return s;
    };

    const prog = gl.createProgram();
    gl.attachShader(prog, compile(gl.VERTEX_SHADER,  VERT_SRC));
    gl.attachShader(prog, compile(gl.FRAGMENT_SHADER, FRAG_SRC));
    gl.linkProgram(prog);
    gl.useProgram(prog);

    // quad covering clip-space [-1,1]
    const buf = gl.createBuffer();
    gl.bindBuffer(gl.ARRAY_BUFFER, buf);
    gl.bufferData(gl.ARRAY_BUFFER, new Float32Array([
        -1,-1, 0,1,
        1,-1, 1,1,
        -1, 1, 0,0,
        1, 1, 1,0
    ]), gl.STATIC_DRAW);

    const aPos = gl.getAttribLocation(prog, 'a_pos');
    const aUV  = gl.getAttribLocation(prog, 'a_uv');
    gl.enableVertexAttribArray(aPos);
    gl.enableVertexAttribArray(aUV);
    gl.vertexAttribPointer(aPos, 2, gl.FLOAT, false, 16, 0);
    gl.vertexAttribPointer(aUV,  2, gl.FLOAT, false, 16, 8);

    // uniforms
    const uDither = gl.getUniformLocation(prog, 'u_dither');
    const uZoom  = gl.getUniformLocation(prog, 'u_zoom');
    const uBayer = gl.getUniformLocation(prog, 'u_bayer[0]');

```

```

gl.uniform1fv(uBayer, new Float32Array(BAYER16));

// texture
const tex = gl.createTexture();
gl.bindTexture(gl.TEXTURE_2D, tex);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.CLAMP_TO_EDGE);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.CLAMP_TO_EDGE);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.NEAREST);

// — INSTANT PLACEHOLDER —————
// Upload a 1x1 Cappella-red pixel IMMEDIATELY so render() has valid
// texture data from frame 0, and the canvas never appears transparent
// (which would let the raw photo bleed through the canvas overlay).
canvasEl.width = 4;
canvasEl.height = 4;
gl.texImage2D(
  gl.TEXTURE_2D, 0, gl.RGBA, 1, 1, 0,
  gl.RGBA, gl.UNSIGNED_BYTE,
  new Uint8Array([209, 32, 47, 255]) // #D1202F
);
gl.clearColor(0.82, 0.125, 0.184, 1.0);
gl.clear(gl.COLOR_BUFFER_BIT);
// —————

let ditherAmt = 1.0;
let zoom = 1.0;
let rafId = null;

const render = () => {
  gl.viewport(0, 0, canvasEl.width, canvasEl.height);
  gl.uniform1f(uDither, ditherAmt);
  gl.uniform1f(uZoom, zoom);
  gl.drawArrays(gl.TRIANGLE_STRIP, 0, 4);
};

// smooth ease helper (mimics GSAP power2.out)
const ease = t => 1 - Math.pow(1 - t, 3);

const tween = (fromD, toD, fromZ, toZ, dur) => {
  if (rafId) cancelAnimationFrame(rafId);
  const start = performance.now();
  const step = now => {
    const t = Math.min((now - start) / dur, 1);
    const et = ease(t);
    ditherAmt = fromD + (toD - fromD) * et;
    zoom = fromZ + (toZ - fromZ) * et;
    render();
    if (t < 1) rafId = requestAnimationFrame(step);
  };
  rafId = requestAnimationFrame(step);
};

// Load image into texture then do first render
const img = new Image();
img.crossOrigin = 'anonymous';
img.onload = () => {
  // KEY DIFFERENCE vs Canvas 2D: downsample to ~35% of natural size.
  // CSS width:100% stretches the low-res canvas to fill the card —
  // each 4x4 Bayer cell then covers ~10 screen pixels, giving the
  // same chunky halftone look as the OCI website (which uses ~40%).
  const scale = 0.35;
  canvasEl.width = Math.max(60, Math.round((img.naturalWidth || 300) * scale));
  canvasEl.height = Math.max(60, Math.round((img.naturalHeight || 300) * scale));
  gl.bindTexture(gl.TEXTURE_2D, tex);
  gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE, img);
  render();
};
img.src = imgUrl;

```

```

return {
  // Reveal full-colour image (like OCI showOriginal)
  showOriginal() { tween(ditherAmt, 0.0, zoom, 1.05, 700); },
  // Restore dither (like OCI showDither)
  showDither() { tween(ditherAmt, 1.0, zoom, 1.00, 700); },
  render
};
}

// --- Per-card WebGL setup ---
cardData.forEach(data => {
  const img = findElByClass(data.imgCls);
  const label = [...scaler.querySelectorAll('span')].find(e =>
    data.labelKeywords.every(k => (e.textContent || '').includes(k))
  );

  if (img && label) {
    img.style.overflow = 'hidden';
    img.style.position = 'relative';

    // WebGL canvas - fills the card image div
    const ditherCanvas = document.createElement('canvas');
    ditherCanvas.style.cssText = [
      'position:absolute', 'top:0', 'left:0',
      'width:100%', 'height:100%',
      'pointer-events:none',
      'image-rendering:pixelated' // keep the halftone crisp
    ].join(';');
    img.appendChild(ditherCanvas);

    // Extract background image URL from Figma div
    const bgUrl = (() => {
      const bg = window.getComputedStyle(img).backgroundImage;
      if (!bg || bg === 'none') return null;
      return bg.replace(/^url\(["]?\/, '').replace(/["]?$/, '');
    })();

    // Create WebGL dither instance and store on element (like OCI .webglDitherInstance)
    let webglInst = null;
    if (bgUrl) {
      webglInst = createWebGLDither(ditherCanvas, bgUrl);
      if (webglInst) ditherCanvas.webglDitherInstance = webglInst;
    }

    // Arrow button (bottom-right corner slides in on hover)
    const arrowBtn = document.createElement('div');
    arrowBtn.style.cssText = [
      'position:absolute', 'bottom:0', 'right:0',
      'width:55px', 'height:55px',
      'background:#fff',
      'display:flex', 'justify-content:center', 'align-items:center',
      'transition:transform 0.45s cubic-bezier(0.25,1,0.5,1), opacity 0.45s ease',
      'transform:translate(100%,100%)', 'opacity:0'
    ].join(';');
    arrowBtn.innerHTML = `<svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="rgb(209,32,47)" stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"><line x1="7"
y1="17" x2="17" y2="7"></line><polyline points="7 7 17 7 17 17"></polyline></svg>`;
    img.appendChild(arrowBtn);

    cards.push({
      id: data.id, img, label, ditherCanvas, arrowBtn,
      webglInst,
      cities: data.cities, isHovered: false, parallaxY: 0
    });
  }
});

// 3. Setup text scramble on cities and school labels
const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ#%&*<>';

```

```

// Wrap target string in individual span tags for character-by-character layout tracking
const wrapInCharSpans = (el) => {
  // innerText keeps rendered line breaks (nested nodes/<br>) as \n, which
  // textContent drops - rebuilding from textContent glued words together
  const originalText = (el.innerText || el.textContent || '').replace(/\u2028/g, '\n');
  el.innerHTML = '';
  const spans = originalText.split('').map(c => {
    const s = document.createElement('span');
    s.textContent = c;
    s.className = 'cap-char';
    // Keep chars inline: inline-block spans collapse whitespace to zero
    // width and allow line breaks between letters mid-word
    if (/s/.test(c)) s.style.whiteSpace = 'pre-wrap';
    el.appendChild(s);
    return s;
  });
  return spans;
};

// Keep the red interlocking C logo perfectly positioned to the right of the active city name
// ROOT CAUSE FIX: hide the logo immediately so it never shows at its default Figma
// position (left:309, top:4464) which overlaps the 'Hyderabad' text. Only show it
// after alignLogo() has successfully re-positioned it.
(() => {
  const logoEl = scaler.querySelector('.fig-asset-eae7f9bdb03982f5-132fb376');
  if (logoEl) {
    logoEl.style.opacity = '0';
    logoEl.style.transition = 'opacity 0.3s ease';
  }
})();

let fontsRegistered = false;
const alignLogo = () => {
  const logoEl = scaler.querySelector('.fig-asset-eae7f9bdb03982f5-132fb376');
  if (!logoEl) return;

  const activeCityEl = cityEls.find(c => c.name === currentCityName)?.el;
  if (!activeCityEl) return;

  // Filter out trailing spaces, line breaks, zero-width chars - only count actual letters
  const charSpans = [...activeCityEl.querySelectorAll('.cap-char')].filter(span => {
    return /[a-zA-Z]/.test(span.textContent || '');
  });

  if (charSpans && charSpans.length > 0) {
    const lastChar = charSpans[charSpans.length - 1];
    const lastCharRect = lastChar.getBoundingClientRect();
    const parentRect = (logoEl.offsetParent || scaler).getBoundingClientRect();
    const s = sc();

    // getBoundingClientRect can return 0-rect for off-screen elements on some browsers.
    // Fall back to reading the span's inline style (Figma coordinates) in that case.
    if (lastCharRect.width === 0 && lastCharRect.height === 0) {
      // Use the city el's own style top/left as a fallback absolute position
      const cityTop = parseFloat(activeCityEl.style.top) || 0;
      const cityLeft = parseFloat(activeCityEl.style.left) || 0;
      const cityWidth = parseFloat(activeCityEl.style.width) || 120;
      const cityHeight = parseFloat(activeCityEl.style.height) || 28;
      logoEl.style.left = (cityLeft + cityWidth + 12) + 'px';
      logoEl.style.top = (cityTop + (cityHeight - 19) / 2) + 'px';
    } else {
      // Position the logo (21px wide, 19px tall) 12px to the right of the last character
      logoEl.style.left = ((lastCharRect.right - parentRect.left) / s + 12) + 'px';
      // Vertically center the logo with the last character
      logoEl.style.top = ((lastCharRect.top - parentRect.top) / s + (lastCharRect.height - 19) / 2) +
'px';
    }
  }

  // Reveal the logo now that it's correctly positioned
  logoEl.style.opacity = '1';
}

```

```

    }

    if (!fontsRegistered) {
      fontsRegistered = true;
      const runAlign = () => {
        alignLogo();
        requestAnimationFrame(alignLogo);
      };
      if (document.fonts) {
        document.fonts.ready.then(runAlign);
      }
      setTimeout(runAlign, 100);
      setTimeout(runAlign, 350);
      setTimeout(runAlign, 800);
      setTimeout(runAlign, 1500);
    }
  };

  const scrambleText = (el, originalText) => {
    // Keep indices aligned with wrapInCharSpans' innerText-based content
    originalText = (el.innerText || originalText).replace(/\u2028/g, '\n');
    const charSpans = wrapInCharSpans(el);
    let interval;
    let step = 0;

    el.addEventListener('mouseenter', () => {
      clearInterval(interval);
      interval = setInterval(() => {
        charSpans.forEach((cs, index) => {
          const char = originalText[index];
          if (char === ' ' || char === '\n' || char === '\u2028') return;
          if (index < step) {
            cs.textContent = char;
          } else {
            cs.textContent = chars[Math.floor(Math.random() * chars.length)];
          }
        });
        alignLogo(); // Align during scramble frames
        step += 1;
        if (step > originalText.length) {
          clearInterval(interval);
          charSpans.forEach((cs, index) => {
            cs.textContent = originalText[index];
          });
          alignLogo();
          step = 0;
        }
      }, 30);
    });

    el.addEventListener('mouseleave', () => {
      clearInterval(interval);
      charSpans.forEach((cs, index) => {
        cs.textContent = originalText[index];
      });
      alignLogo();
      step = 0;
    });
  };

  cityEls.forEach(c => {
    scrambleText(c.el, c.el.textContent || '');
  });

  cards.forEach(c => {
    scrambleText(c.label, c.label.textContent || '');
  });

  // Core layout function combining filter state, hover zoom, and scroll parallax
  const updateCardTransforms = () => {

```

```

cards.forEach(c => {
  const isMatch = c.cities.includes(currentCityName);

  c.img.style.transition = 'transform 0.18s cubic-bezier(0.25, 1, 0.5, 1), opacity 0.4s ease';
  c.label.style.transition = 'transform 0.18s cubic-bezier(0.25, 1, 0.5, 1), opacity 0.4s ease';

  if (isMatch) {
    c.img.style.pointerEvents = 'auto';
    c.img.style.opacity = '1';

    // Don't CSS-scale the card image itself - WebGL handles zoom internally via u_zoom
    c.img.style.transform = `translateY(${c.parallaxY}px)`;
    c.label.style.transform = `translateY(${c.parallaxY}px)`;

    // Trigger WebGL dither tween (mirrors OCI mouseenter/mouseleave exactly)
    if (c.webglInst) {
      if (c.isHovered) {
        c.webglInst.showOriginal(); // uDitherAmount → 0.0, zoom → 1.05
      } else {
        c.webglInst.showDither(); // uDitherAmount → 1.0, zoom → 1.00
      }
    }

    if (c.isHovered) {
      c.arrowBtn.style.opacity = '1';
      c.arrowBtn.style.transform = 'translate(0, 0)';
    } else {
      c.arrowBtn.style.opacity = '0';
      c.arrowBtn.style.transform = 'translate(100%, 100%)';
    }
  } else {
    c.img.style.pointerEvents = 'none';
    c.img.style.opacity = '0';
    c.img.style.transform = 'translateY(30px) scale(0.95)';
    c.label.style.opacity = '0';
    c.label.style.transform = 'translateY(30px)';

    // Always show dither on hidden cards
    if (c.webglInst) c.webglInst.showDither();
    c.arrowBtn.style.opacity = '0';
    c.arrowBtn.style.transform = 'translate(100%, 100%)';
  }
});

// 4. Setup card hover zoom and overlay fade-out
cards.forEach(c => {
  c.img.addEventListener('mouseenter', () => {
    c.isHovered = true;
    updateCardTransforms();
  });

  c.img.addEventListener('mouseleave', () => {
    c.isHovered = false;
    updateCardTransforms();
  });
});

// 5. Setup dynamic city filtering
const filterToCity = (cityName) => {
  currentCityName = cityName;
  cityEls.forEach(c => {
    if (c.name === cityName) {
      c.el.style.color = 'rgb(0,0,0)';
      c.el.style.fontWeight = '700';
    } else {
      c.el.style.color = 'rgb(186,186,186)';
      c.el.style.fontWeight = '500';
    }
  });
});

```



```

    updateCardTransforms();
    alignLogo(); // Align on filter change
  };

  cityEls.forEach(c => {
    c.el.style.cursor = 'pointer';
    c.el.addEventListener('click', () => {
      filterToCity(c.name);
    });
  });

  // 6. Setup scroll parallax listener (sc() already defined at top of method)
  this._onPortfolioScroll = () => {
    const scrollY = window.scrollY;
    const s = sc();
    // Portfolio area center in scaled pixels (approx 4400px)
    const centerOffset = (scrollY + window.innerHeight / 2 - 4400 * s) / s;

    // Calculate parallax offset (Column 1 scrolls at different speed than Column 2)
    const offsetVal = centerOffset * 0.12;

    cards.forEach(c => {
      // Column 1: cards 1 & 3 (left side) -> translate up
      // Column 2: cards 2 & 4 (right side) -> translate down
      const isCol1 = (c.id === 1 || c.id === 3);
      c.parallaxY = isCol1 ? -offsetVal : offsetVal;
    });

    updateCardTransforms();
  };

  window.addEventListener('scroll', this._onPortfolioScroll, { passive: true });

  // Track layout resize shifts
  this._alignLogo = alignLogo;
  window.addEventListener('resize', this._alignLogo);

  // Default to Hyderabad on load
  filterToCity('Hyderabad');
  this._onPortfolioScroll();

  // Re-run hide overlays after card DOM settles (belt and suspenders)
  setTimeout(hideRedOverlays, 800);
}

```

14 — “Cappella Edge” accordion

`_setupAccordion()` · lines 2908–3240

Where: The four-row accordion in “The Cappella Edge” section.

How: Rows are absolutely positioned, so opening item *i* recomputes every top below it with a shared 0.65s power-curve transition. Per item: hover roll, plus icon rotating 135° to ×, masked description rising from `translateY(110%)`, and a Bayer dither flash on the section photo at every change.

Full function — `_setupAccordion()` (lines 2908–3240)

```
_setupAccordion() {
  const scaler = document.getElementById('cap-scaler');
  if (!scaler) return;

  const sc = () => {
    const m = (scaler.style.transform || '').match(/scale\(((^)+)\)/);
    return m ? parseFloat(m[1]) : 1;
  };

  // Geometry measured from the baked Figma frame, which ships with item 3
  // ("Active asset management") open — everything below its description sits
  // 258px lower than its closed spot. All values here are CLOSED-state
  // coordinates. Opening item i pushes the rows/dividers below it down by
  // that item's openH. The baked open state is the tallest layout the
  // section reaches, so content below the section never needs to move.
  const DESC_LEFT = 755, DESC_WIDTH = 624, DESC_OFFSET = 62;
  const ICON_OFFSET = 20;
  const EASE = 'cubic-bezier(0.76, 0, 0.24, 1)'; // OCI power curve
  const BAKED_OPEN_INDEX = 2, BAKED_SHIFT = 258;

  const data = [
    {
      title: 'Operator-aligned structures',
      text: 'Flexible models — PropCo+OpCo, Sale & Leaseback, Greenfield',
      closedTop: 2315, figmaIconTop: 2335, closedDividerTop: 2386
    },
    {
      title: 'Future-ready campuses',
      text: 'Purpose-built and revamped campuses with modern facilities, green spaces, and scalable infrastructure.',
      closedTop: 2389, figmaIconTop: 2409, closedDividerTop: 2452
    },
    {
      title: 'Active asset management',
      text: 'Through our ecosystem affiliates, we integrate operations, value-added services, and construction expertise to create a high-performance, future-ready institution from day one.',
      closedTop: 2455, figmaIconTop: 2487, closedDividerTop: 2534, figmaShift: BAKED_SHIFT
    },
    {
      title: 'Proven, long-term investor value',
      text: 'A strong track record of delivering consistent, high-quality returns across three funds.',
      closedTop: 2537, figmaIconTop: 2817, closedDividerTop: 2603
    }
  ];

  // Hide the baked Figma +/- icons (replaced with animatable ones below)
  data.forEach(d => {
    const oldIcon = [...scaler.querySelectorAll('div')].find(e => {
      const style = e.getAttribute('style') || '';
      return style.includes('left: 1388') && style.includes('top: ' + d.figmaIconTop);
    }) || [...scaler.querySelectorAll('svg')].find(e => {
      const style = e.getAttribute('style') || '';
      return style.includes('matrix') && style.includes('1410') && style.includes(' ' + d.figmaIconTop);
    });
  });
```

```

    if (oldIcon) oldIcon.style.display = 'none';
  });

  // Hide the baked open description (recreated below with a masked reveal)
  const bakedDesc = [...scaler.querySelectorAll('span')].find(e =>
    (e.textContent || '').includes('Through our ecosystem')
  );
  if (bakedDesc) bakedDesc.style.display = 'none';

  // Divider lines are 691x1 SVGs with transform matrix(-1,0,0,-1,1426,ty)
  const allSVGs = [...scaler.querySelectorAll('svg')];
  const findDivider = (bakedTy) => allSVGs.find(e => {
    if (e.getAttribute('width') !== '691') return false;
    const m = (e.getAttribute('style') || '').match(/matrix\([^,]+,[^,]+,[^,]+,[^,]+\s*1426,\s*([-\\d.]+)\)/);
    return m && Math.abs(parseFloat(m[1]) - bakedTy) < 8;
  });

  const items = [];
  data.forEach((d, idx) => {
    const titleEl = [...scaler.querySelectorAll('span')].find(e => {
      const txt = (e.textContent || '').trim().replace(/\\xA0/g, ' ').replace(/\\s+/g, ' ');
      return txt.includes(d.title);
    });
    if (!titleEl) return;

    // OCI-style hover roll: duplicate the title inside an overflow mask
    const rowH = titleEl.getBoundingClientRect().height / sc();
    titleEl.style.height = rowH + 'px';
    titleEl.style.overflow = 'hidden';
    titleEl.style.transition = 'top 0.65s ' + EASE;
    const roll = document.createElement('span');
    roll.style.cssText = 'display:block; transition: transform 0.45s ' + EASE + ' ';
    const line1 = document.createElement('span');
    line1.style.display = 'block';
    line1.textContent = d.title;
    const line2 = line1.cloneNode(true);
    roll.appendChild(line1);
    roll.appendChild(line2);
    titleEl.textContent = '';
    titleEl.appendChild(roll);

    // Plus icon - styled as thin + bars, rotates to x via GSAP on open (OCI signature)
    const iconContainer = document.createElement('div');
    iconContainer.style.cssText =
      'position:absolute; left:1388px; width:22px; height:22px;' +
      'display:flex; align-items:center; justify-content:center;' +
      'color:rgb(209,32,47); pointer-events:none;' +
      'transition: top 0.65s ' + EASE + ' ';
    const hBar = document.createElement('div');
    hBar.style.cssText = 'position:absolute; width:22px; height:2px; background:currentColor;';
    const vBar = document.createElement('div');
    vBar.style.cssText = 'position:absolute; width:2px; height:22px; background:currentColor;';
    iconContainer.appendChild(hBar);
    iconContainer.appendChild(vBar);
    scaler.appendChild(iconContainer);

    // Masked description + [ LEARN MORE → ] link (OCI panel content)
    const descWrap = document.createElement('div');
    descWrap.style.cssText =
      'position:absolute; left:' + DESC_LEFT + 'px; width:' + DESC_WIDTH + 'px;' +
      'overflow:hidden; pointer-events:none; transition: top 0.65s ' + EASE + ' ';
    const inner = document.createElement('div');
    inner.style.cssText =
      'transform: translateY(110%); opacity:0;' +
      'transition: transform 0.7s ' + EASE + ', opacity 0.45s ease;';
    const txt = document.createElement('div');
    txt.style.cssText =
      'font-family: Montserrat, sans-serif; font-weight:300; font-size:24px;' +
      'line-height:123.131%; color: rgb(0,0,0); white-space: normal;';
  });

```

```

txt.textContent = d.text;
const link = document.createElement('div');
link.style.cssText =
  'margin-top:22px; display:inline-flex; align-items:center; gap:8px;' +
  'font-family: Montserrat, sans-serif; font-weight:600; font-size:13px;' +
  'letter-spacing:1.5px; color: rgb(209,32,47); cursor:pointer; pointer-events:auto;';
const arrow = document.createElement('span');
arrow.textContent = '→';
arrow.style.cssText = 'display:inline-block; transition: transform 0.35s ' + EASE + ' ';
link.appendChild(document.createTextNode(' LEARN MORE '));
link.appendChild(arrow);
link.appendChild(document.createTextNode(' '));
link.addEventListener('mouseenter', () => { arrow.style.transform = 'translateX(6px)'; });
link.addEventListener('mouseleave', () => { arrow.style.transform = 'translateX(0)'; });
link.addEventListener('click', () => {
  const vp = document.getElementById('cap-viewport');
  const target = 6942 * ((vp ? vp.clientWidth : window.innerWidth) / 1440);
  if (this._lenis) this._lenis.scrollTo(target);
  else window.scrollTo({ top: target, behavior: 'smooth' });
});
inner.appendChild(txt);
inner.appendChild(link);
descWrap.appendChild(inner);
scaler.appendChild(descWrap);

// Freeze the mask to the content's natural height, derive the open size
const innerH = Math.ceil(inner.getBoundingClientRect().height / sc());
descWrap.style.height = innerH + 'px';
const openH = Math.max(d.figmaShift || 0, DESC_OFFSET + innerH + 28);

// Divider under this row (item 3's is missing in Figma – clone one)
const bakedDividerTy = d.closedDividerTop + (idx >= BAKED_OPEN_INDEX ? BAKED_SHIFT : 0);
let dividerEl = findDivider(bakedDividerTy);
if (!dividerEl && idx === BAKED_OPEN_INDEX) {
  const donor = findDivider(2386);
  if (donor) {
    dividerEl = donor.cloneNode(true);
    donor.parentNode.appendChild(dividerEl);
  }
}
const setDividerTop = dividerEl
  ? (ty) => { dividerEl.style.transform = 'matrix(-1, 0, 0, -1, 1426, ' + ty + ')'; }
  : () => {};
if (dividerEl) dividerEl.style.transition = 'transform 0.65s ' + EASE;

// Full-row click target – covers entire accordion row from title to icon
const hitArea = document.createElement('div');
hitArea.style.cssText =
  'position:absolute; left:150px; width:1280px; height:72px;' +
  'cursor:pointer; z-index:10; transition: top 0.65s ' + EASE + ' ';
scaler.appendChild(hitArea);

items.push({ ...d, titleEl, roll, rowH, iconContainer, descWrap, inner, openH, dividerEl,
setDividerTop, hitArea });
});

if (!items.length) return;

let activeIndex = -1;
const layout = () => {
  let shift = 0;
  items.forEach((item, i) => {
    const isOpen = i === activeIndex;
    const top = item.closedTop + shift;
    item.titleEl.style.top = top + 'px';
    item.iconContainer.style.top = (top + ICON_OFFSET) + 'px';

    // OCI-exact GSAP rotation: 135deg open (*), 0deg closed
    if (window.gsap) {
      window.gsap.to(item.iconContainer, {

```

```

        rotation: isOpen ? 135 : 0,
        duration: 0.8,
        ease: 'power4.inOut',
        overwrite: true
    });
} else {
    item.iconContainer.style.transform = isOpen ? 'rotate(135deg)' : 'rotate(0deg)';
}

item.descWrap.style.top = (top + DESC_OFFSET) + 'px';
item.hitArea.style.top = (top - 4) + 'px';
if (isOpen) {
    item.inner.style.transitionDelay = '0.15s, 0.15s';
    item.inner.style.transform = 'translateY(0)';
    item.inner.style.opacity = '1';
    shift += item.openH;
} else {
    item.inner.style.transitionDelay = '0s, 0s';
    item.inner.style.transform = 'translateY(110%)';
    item.inner.style.opacity = '0';
}
item.setDividerTop(item.closedDividerTop + shift);
});
};

// Dithered flash on the section image whenever the accordion changes (OCI)
const edgeImg = scaler.querySelector('.fig-asset-63186349250e044c-069f3173');
let flashImage = () => {};
if (edgeImg) {
    edgeImg.style.overflow = 'hidden';
    edgeImg.style.transition = 'transform 0.6s ' + EASE;
    const canvas = document.createElement('canvas');
    canvas.style.cssText =
        'position:absolute; left:0; top:0; width:100%; height:100%;' +
        'pointer-events:none; opacity:0; transition: opacity 0.4s ease;';
    edgeImg.appendChild(canvas);

    const bayer = [[0,12,3,15],[8,4,11,7],[2,14,1,13],[10,6,9,5]];
    const bg = window.getComputedStyle(edgeImg).backgroundImage;
    if (bg && bg !== 'none') {
        const tempImg = new Image();
        tempImg.crossOrigin = 'anonymous';
        tempImg.src = bg.replace(/^url\(["']?/, '').replace(/["']?\)$/ , '');
        tempImg.onload = () => {
            const w = 150, h = Math.round((tempImg.height / tempImg.width) * w);
            canvas.width = w; canvas.height = h;
            const ctx = canvas.getContext('2d');
            ctx.drawImage(tempImg, 0, 0, w, h);
            const imgData = ctx.getImageData(0, 0, w, h);
            const px = imgData.data;
            for (let y = 0; y < h; y++) {
                for (let x = 0; x < w; x++) {
                    const i4 = (y * w + x) * 4;
                    const gray = 0.299 * px[i4] + 0.587 * px[i4 + 1] + 0.114 * px[i4 + 2];
                    const light = gray > ((bayer[y % 4][x % 4] + 0.5) / 16) * 255;
                    // Brand red duotone: dark=rgb(209,32,47) red, light=rgb(255,235,235) pink
                    px[i4] = light ? 255 : 209;
                    px[i4 + 1] = light ? 235 : 32;
                    px[i4 + 2] = light ? 235 : 47;
                }
            }
            ctx.putImageData(imgData, 0, 0);
        };
    }

    let flashT = null;
    flashImage = () => {
        canvas.style.opacity = '1';
        edgeImg.style.transform = 'scale(1.03)';
        clearTimeout(flashT);
    }
}

```

```

        flashT = setTimeout(() => {
            canvas.style.opacity = '0';
            edgeImg.style.transform = 'scale(1)';
        }, 420);
    };
}

items.forEach((item, i) => {
    item.hitArea.addEventListener('click', () => {
        activeIndex = activeIndex === i ? -1 : i;
        layout();
        flashImage();
    });
    // OCI hover: pre-rotate to 90° when hovered (closed), back to 0° on leave
    item.hitArea.addEventListener('mouseenter', () => {
        item.roll.style.transform = 'translateY(-' + item.rowH + 'px)';
        const isOpen = activeIndex === i;
        if (!isOpen && window.gsap) {
            window.gsap.to(item.iconContainer, {
                rotation: 90,
                duration: 0.3,
                ease: 'power4.inOut',
                overwrite: true
            });
        }
    });
    item.hitArea.addEventListener('mouseleave', () => {
        item.roll.style.transform = 'translateY(0)';
        const isOpen = activeIndex === i;
        if (!isOpen && window.gsap) {
            window.gsap.to(item.iconContainer, {
                rotation: 0,
                duration: 0.3,
                ease: 'power4.inOut',
                overwrite: true
            });
        }
    });
});

// Apply the baked open state without animating the initial layout...
const frozen = [];
items.forEach(item => {
    [item.titleEl, item.iconContainer, item.descWrap, item.hitArea, item.dividerEl].forEach(el => {
        if (!el) return;
        frozen.push([el, el.style.transition]);
        el.style.transition = 'none';
    });
});
activeIndex = BAKED_OPEN_INDEX;
layout();
// ...but keep the open description hidden so it can slide in on scroll
const openItem = items[activeIndex];
if (openItem) {
    openItem.inner.style.transform = 'translateY(110%)';
    openItem.inner.style.opacity = '0';
}
requestAnimationFrame(() => {
    frozen.forEach(([el, t]) => { el.style.transition = t; });
    if (openItem) {
        const io = new IntersectionObserver((entries) => {
            entries.forEach(entry => {
                if (entry.isIntersecting) {
                    openItem.inner.style.transform = 'translateY(0)';
                    openItem.inner.style.opacity = '1';
                    io.disconnect();
                }
            });
        });
        io.observe(openItem.descWrap);
    }
});

```

```
}  
});  
}
```

15 — Footer: curtain reveal + staged rise + live clock

`_setupFooterAnimations()` · lines 1466-1601

`.cap-ft-link CSS` · lines 433-447

Where: The red footer.

How: The footer becomes an overflow-hidden window; its content starts at y:-420 and scrubs to 0 as the footer scrolls in (curtain uncover). The big C logo wipes in via clip-path, everything else fades with column-keyed delays, a Hyderabad clock ticks every second, and links get an underline wipe.

CSS — footer underline wipe (lines 433-447)

```
/* ===== FOOTER PREMIUM ===== */
.cap-ft-link { position: relative; }
.cap-ft-link::after {
  content: '';
  position: absolute;
  left: 0;
  bottom: -3px;
  width: 100%;
  height: 1px;
  background: rgb(255, 255, 255);
  transform: scaleX(0);
  transform-origin: left;
  transition: transform 0.45s var(--cap-ease-power);
}
.cap-ft-link:hover::after { transform: scaleX(1); }
```

Full function — `_setupFooterAnimations()` (lines 1466-1601)

```
_setupFooterAnimations() {
  const scaler = document.getElementById('cap-scaler');
  if (!scaler || this._footerAnimated) return;
  this._footerAnimated = true;
  const reduced = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
  const m = (scaler.style.transform || '').match(/scale\(((^)+)\)/);
  const scale = m ? parseFloat(m[1]) : 1;
  const scalerRect = scaler.getBoundingClientRect();
  const frameBox = (el) => {
    const r = el.getBoundingClientRect();
    return { top: (r.top - scalerRect.top) / scale, left: (r.left - scalerRect.left) / scale,
      w: r.width / scale, h: r.height / scale };
  };
  const hasMatrix = (el) => ((el.getAttribute('style') || '').includes('matrix'));

  // ===== FOOTER REVEAL =====
  const footer = [...scaler.querySelectorAll('div')].find(d => d.getAttribute('data-original-top') ===
'7092');
  if (!footer) return;
  const footerTopFrame = parseFloat(footer.style.top) || 7092;

  // Live local-time line, OCI-editorial style
  const clock = document.createElement('div');
  clock.style.cssText =
    'position:absolute; left:1117px; top:' + (footerTopFrame + 26) + 'px; z-index:5;' +
    'font-family:"Space Mono", monospace; font-size:12px; letter-spacing:2px;' +
    'color:rgba(255,255,255,0.75); white-space:nowrap;';
  const tickClock = () => {
    clock.textContent = 'HYDERABAD \u2014 ' +
      new Date().toLocaleTimeString('en-IN', { timeZone: 'Asia/Kolkata', hour12: false }) + ' IST';
  };
  tickClock();
  this._clockIv = setInterval(tickClock, 1000);
  scaler.appendChild(clock);
}
```



```

// Underline-wipe hover on footer links
const linkTexts = ['ABOUT US', 'PORTFOLIO', 'TEAM', 'CONTACT US', 'LINKEDIN', 'PARTNER@CAPPELLA.IN'];
[...footer.querySelectorAll('span')].forEach(sp => {
  const t = (sp.textContent || '').trim().toUpperCase();
  if (linkTexts.includes(t)) {
    sp.classList.add('cap-ft-link');
    sp.style.cursor = 'pointer';
  }
});

// Email input focus states + arrow nudge
const emailInput = footer.querySelector('input[type="email"]');
if (emailInput) {
  emailInput.style.transition = 'border-color 0.4s ease';
  emailInput.addEventListener('focus', () => { emailInput.style.borderBottom = '1px solid
rgba(255,255,255,0.95)'; });
  emailInput.addEventListener('blur', () => { emailInput.style.borderBottom = '1px solid
rgba(255,255,255,0.4)'; });
}
const arrow = [...footer.querySelectorAll('svg')].find(sv => {
  const l = parseFloat(sv.style.left);
  return l >= 1300 && l <= 1330;
});
if (arrow) {
  arrow.style.transition = 'transform 0.35s var(--cap-ease-power)';
  arrow.addEventListener('mouseenter', () => { arrow.style.transform = 'translateX(5px)'; });
  arrow.addEventListener('mouseleave', () => { arrow.style.transform = 'translateX(0)'; });
}

if (reduced) return;

// Stage every footer element: big C logo wipes in, the rest rise
// column-by-column (delay keyed to their position in the footer)
const bigLogo = footer.querySelector('.fig-asset-c9d8f12cf6e90c99');
const kids = [...footer.children].filter(el => {
  if (parseFloat(el.style.width) > 1400) return false; // red background slab
  return true;
});
const staged = [];
kids.forEach(el => {
  const b = frameBox(el);
  const relTop = b.top - footerTopFrame;
  const delay = 0.15 + (b.left / 1440) * 0.4 + Math.max(0, relTop / 530) * 0.15;
  if (el === bigLogo) {
    el.style.clipPath = 'inset(0 100% 0 0)';
    el.style.transition = 'clip-path 1.1s var(--cap-ease-expo) 0.1s';
    staged.push(() => { el.style.clipPath = 'inset(0 0% 0 0)'; });
  } else {
    // opacity-only: the curtain scrub below owns the transform channel
    el.style.opacity = '0';
    el.style.transition = 'opacity 0.85s var(--cap-ease-expo) ' + delay + 's';
    staged.push(() => { el.style.opacity = '1'; });
  }
});
clock.style.opacity = '0';
clock.style.transition = 'opacity 0.85s var(--cap-ease-expo) 0.5s';
staged.push(() => { clock.style.opacity = '1'; });

const io = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (!entry.isIntersecting) return;
    io.disconnect();
    staged.forEach(fn => fn());
  });
}, { threshold: 0.25 });
io.observe(footer);

// ===== CURTAIN REVEAL =====
// The footer container becomes a clipped window; its content is offset
// one footer-height up and scrubs to rest as the window scrolls in, so

```

```

// the footer reads as a fixed surface being uncovered by the page.
if (window.gsap && window.ScrollTrigger) {
  footer.style.overflow = 'hidden';
  clock.style.zIndex = '6';
  // Capture slide targets AFTER the deferred footer-email setup (runs
  // ~500ms later) replaces the placeholder span with the real input -
  // otherwise the input is left out and stays static during the scrub
  setTimeout(() => {
    const slide = [...footer.children].filter(el => !(parseFloat(el.style.width) > 1400));
    // Part of the footer's content (email, address, designed-by, partner
    // logos) lives OUTSIDE the container in the Figma DOM - without them
    // the curtain shears the footer apart mid-scrub. adjustFooter tagged
    // everything in the footer region with data-original-top; take the
    // top-most tagged elements that aren't inside the container.
    const tagged = [...scaler.querySelectorAll('[data-original-top]')].filter(el => {
      if (el === footer || footer.contains(el)) return false;
      return parseFloat(el.getAttribute('data-original-top')) > 7092;
    });
    tagged.filter(el => !tagged.some(a => a !== el && a.contains(el)))
      .forEach(el => slide.push(el));
    slide.push(clock); // static clock would overlap sliding text mid-scrub
    window.gsap.fromTo(slide, { y: -420 }, {
      y: 0,
      ease: 'none',
      scrollTrigger: {
        trigger: footer,
        start: 'top bottom',
        end: 'bottom bottom',
        scrub: true
      }
    });
  }, 700);
}

```

16 — Operators logo marquee

`_setupOperatorsMarquee()` · lines 1603-1690

Where: The operator logos section — two bands scrolling in opposite directions.

How: Logos are cloned into a repeated flex track translated by a shared rAF loop, wrapping via modulo on the sequence width. Speed eases between 34 px/s and 0 on hover; logos nudge up on hover; the loop idles off-screen.

Full function — `_setupOperatorsMarquee()` (lines 1603-1690)

```
_setupOperatorsMarquee() {
  const scaler = document.getElementById('cap-scaler');
  if (!scaler || this._mqBuilt) return;
  this._mqBuilt = true;
  if (window.matchMedia('(prefers-reduced-motion: reduce)').matches) return;

  // Operator logos (fig-asset background divs) measured from the frame
  const ROW_A = ['fig-asset-79356d431474ada0', 'fig-asset-91ee4f07eflea5a8',
    'fig-asset-620dc3d6704178dd', 'fig-asset-1ec5ca333db94fc0'];
  const ROW_B = ['fig-asset-1e639ffbc0aca215', 'fig-asset-1462c629c4c2879d',
    'fig-asset-55a8dd7666421f70-d65f9521'];
  const rowAEls = ROW_A.map(c => scaler.querySelector('.' + CSS.escape(c))).filter(Boolean);
  const rowBEls = ROW_B.map(c => scaler.querySelector('.' + CSS.escape(c))).filter(Boolean);
  if (rowAEls.length < 3 || rowBEls.length < 2) return; // frame changed — keep the grid

  // Hide the static grid + the triangle decorations woven between the rows
  [...rowAEls, ...rowBEls].forEach(el => { el.style.display = 'none'; });
  [...scaler.querySelectorAll('div')].forEach(el => {
    const st = el.getAttribute('style') || '';
    const top = parseFloat(el.style.top) || 0;
    if (top > 5560 && top < 6900 && st.includes('linear-gradient') && st.includes('209, 32, 47')) {
      el.style.display = 'none';
    }
  });

  // Build a marquee band: [row of cells]x(repeats), translated on rAF
  const rows = [];
  const buildBand = (els, top, dir) => {
    const band = document.createElement('div');
    band.style.cssText =
      'position:absolute; left:0; top:' + top + 'px; width:1440px; height:170px;' +
      'overflow:hidden; z-index:3; opacity:0; transition: opacity 1s var(--cap-ease-expo)';
    const track = document.createElement('div');
    track.style.cssText =
      'display:flex; align-items:center; height:100%; width:max-content;' +
      'will-change:transform';
    const GAP = 150;
    const seqW = els.reduce((sum, el) => sum + (parseFloat(el.style.width) || 180) + GAP, 0);
    const repeats = Math.max(3, Math.ceil((1440 + seqW) / seqW) + 1);
    for (let rep = 0; rep < repeats; rep++) {
      els.forEach(el => {
        const cell = el.cloneNode(true);
        cell.style.display = 'block';
        cell.style.position = 'static';
        cell.style.left = 'auto';
        cell.style.top = 'auto';
        cell.style.transform = 'none';
        cell.style.margin = '0 ' + (GAP / 2) + 'px';
        cell.style.flex = '0 0 auto';
        cell.style.transition = 'transform 0.5s var(--cap-ease-expo)';
        cell.addEventListener('mouseenter', () => { cell.style.transform = 'translateY(-6px)'; });
        cell.addEventListener('mouseleave', () => { cell.style.transform = 'translateY(0)'; });
        track.appendChild(cell);
      });
    }
  };
}
```

```

    }
    band.appendChild(track);
    scaler.appendChild(band);
    const row = { band, track, seqW, dir, x: 0, speed: 0, targetSpeed: 34 };
    band.addEventListener('mouseenter', () => { row.targetSpeed = 0; });
    band.addEventListener('mouseleave', () => { row.targetSpeed = 34; });
    const io = new IntersectionObserver((entries) => {
      entries.forEach(entry => { if (entry.isIntersecting) { band.style.opacity = '1'; io.disconnect(); }
    });
    }, { threshold: 0.3 });
    io.observe(band);
    rows.push(row);
  };
  buildBand(rowAEls, 5810, -1);
  buildBand(rowBEls, 6330, 1);

  // Shared drive loop - skips work while both bands are off-screen
  let lastT = performance.now();
  const step = (now) => {
    const dt = Math.min((now - lastT) / 1000, 0.05);
    lastT = now;
    let visible = false;
    rows.forEach(row => {
      const r = row.band.getBoundingClientRect();
      if (r.bottom < -100 || r.top > window.innerHeight + 100) return;
      visible = true;
      row.speed += (row.targetSpeed - row.speed) * 0.06; // eased pause/resume
      row.x = (row.x + row.dir * row.speed * dt) % row.seqW;
      const offset = row.dir === -1 ? row.x : row.x - row.seqW;
      row.track.style.transform = 'translateX(' + offset.toFixed(2) + 'px)';
    });
    this._mqRaf = requestAnimationFrame(step);
  };
  this._mqRaf = requestAnimationFrame(step);
}

```

17 — Menu overlay

CSS · lines 261-431

_setupMenuOverlay() · lines 1762-1880

Where: Floating hamburger button (appears after ~180 px of scroll) and the slide-in side panel.

How: Class-toggle driven: .cap-menu-open slides the panel in from translateX(102%), fades the backdrop, and the big links rise out of overflow masks with per-index transition delays set in JS.

CSS — button, backdrop, panel, links, rows, email (lines 261-431)

```
/* ===== OCI-STYLE MENU OVERLAY ===== */
#cap-menu-btn {
  position: fixed;
  top: 32px;
  right: 32px;
  z-index: 94;
  width: 52px;
  height: 52px;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.85);
  backdrop-filter: blur(8px);
  -webkit-backdrop-filter: blur(8px);
  border: 1px solid rgba(209, 32, 47, 0.25);
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.08);
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  user-select: none;
  opacity: 0;
  pointer-events: none;
  transition: transform 0.3s cubic-bezier(0.16, 1, 0.3, 1), background-color 0.3s ease, border-color
0.3s ease, opacity 0.3s ease;
}
#cap-menu-btn:hover {
  background: rgb(209, 32, 47);
  border-color: rgb(209, 32, 47);
  transform: scale(1.08);
}
#cap-menu-btn svg {
  width: 20px;
  height: 14px;
  transition: stroke 0.3s ease;
}
#cap-menu-btn:hover svg {
  stroke: rgb(255, 255, 255) !important;
}

#cap-menu-overlay {
  position: fixed;
  inset: 0;
  z-index: 93;
  visibility: hidden;
  pointer-events: none;
}
#cap-menu-overlay.cap-menu-open { visibility: visible; pointer-events: auto; }
#cap-menu-backdrop {
  position: absolute;
  inset: 0;
  background: rgba(120, 12, 22, 0.35);
  opacity: 0;
  transition: opacity 0.6s var(--cap-ease-power);
}
.cap-menu-open #cap-menu-backdrop { opacity: 1; }
#cap-menu-panel {
```

```

position: absolute;
top: 0;
right: 0;
height: 100%;
width: min(600px, 94vw);
background: rgb(255, 255, 255);
border-left: 1px solid rgba(209, 32, 47, 0.25);
transform: translateX(102%);
transition: transform 0.75s var(--cap-ease-power);
display: flex;
flex-direction: column;
padding: 26px 44px 34px;
box-sizing: border-box;
overflow-y: auto;
}
.cap-menu-open #cap-menu-panel { transform: translateX(0); }

.cap-menu-top {
display: flex;
justify-content: space-between;
align-items: center;
margin-bottom: 56px;
}
.cap-menu-tag {
font-family: 'Space Mono', monospace;
font-size: 12px;
letter-spacing: 2px;
color: rgba(209, 32, 47, 0.75);
}
#cap-menu-close {
font-family: 'Space Mono', monospace;
font-size: 12px;
letter-spacing: 2px;
color: rgb(209, 32, 47);
background: rgb(255, 255, 255);
border: 1px solid rgba(209, 32, 47, 0.35);
padding: 12px 20px;
cursor: pointer;
transition: background 0.35s var(--cap-ease-power), color 0.35s var(--cap-ease-power);
}
#cap-menu-close:hover { background: rgb(209, 32, 47); color: rgb(255, 255, 255); }

.cap-menu-links { display: flex; flex-direction: column; gap: 6px; margin-bottom: auto; }
.cap-menu-links .cap-menu-mask { overflow: hidden; }
.cap-menu-links a {
display: inline-block;
font-family: Montserrat, sans-serif;
font-weight: 600;
font-size: 48px;
line-height: 1.25;
color: rgb(209, 32, 47);
cursor: pointer;
transform: translateY(115%);
transition: transform 0.8s var(--cap-ease-expo), padding-left 0.45s var(--cap-ease-power);
}
.cap-menu-open .cap-menu-links a { transform: translateY(0); }
.cap-menu-links a:hover { padding-left: 18px; }

.cap-menu-row {
display: flex;
justify-content: space-between;
gap: 24px;
border-top: 1px solid rgba(209, 32, 47, 0.3);
padding: 18px 0;
opacity: 0;
transform: translateY(16px);
transition: opacity 0.7s var(--cap-ease-expo), transform 0.7s var(--cap-ease-expo);
}
.cap-menu-open .cap-menu-row { opacity: 1; transform: translateY(0); }
.cap-menu-label {

```

```

    font-family: 'Space Mono', monospace;
    font-size: 12px;
    letter-spacing: 2px;
    color: rgb(209, 32, 47);
    padding-top: 3px;
    white-space: nowrap;
}
.cap-menu-info {
    text-align: right;
    font-family: Montserrat, sans-serif;
    font-weight: 400;
    font-size: 15px;
    line-height: 1.6;
    color: rgb(40, 40, 40);
}
.cap-menu-info a { color: rgb(209, 32, 47); cursor: pointer; display: inline-block; }
.cap-menu-info a:hover { color: rgb(170, 20, 34); }

.cap-menu-email {
    display: flex;
    border: 1px solid rgba(209, 32, 47, 0.4);
    margin-top: 18px;
    opacity: 0;
    transform: translateY(16px);
    transition: opacity 0.7s var(--cap-ease-expo) 0.1s, transform 0.7s var(--cap-ease-expo) 0.1s;
}
.cap-menu-open .cap-menu-email { opacity: 1; transform: translateY(0); }
.cap-menu-email input {
    flex: 1;
    border: none;
    outline: none;
    background: transparent;
    padding: 18px 20px;
    font-family: Montserrat, sans-serif;
    font-size: 15px;
    color: rgb(40, 40, 40);
}
.cap-menu-email button {
    border: none;
    background: transparent;
    color: rgb(209, 32, 47);
    font-size: 22px;
    padding: 0 22px;
    cursor: pointer;
    transition: transform 0.35s var(--cap-ease-power);
}
.cap-menu-email button:hover { transform: translateX(5px); }

```

Full function — `_setupMenuOverlay()` (lines 1762-1880)

```

_setupMenuOverlay() {
    if (document.getElementById('cap-menu-btn')) return;

    const btn = document.createElement('div');
    btn.id = 'cap-menu-btn';
    btn.innerHTML = `
        <svg viewBox="0 0 20 14" fill="none" stroke="rgb(209, 32, 47)" stroke-width="2.5" stroke-
linecap="round">
            <line x1="0" y1="1.25" x2="20" y2="1.25"></line>
            <line x1="0" y1="7" x2="20" y2="7"></line>
            <line x1="0" y1="12.75" x2="20" y2="12.75"></line>
        </svg>
    `;
    document.body.appendChild(btn);

    const overlay = document.createElement('div');
    overlay.id = 'cap-menu-overlay';
    overlay.innerHTML =
        '<div id="cap-menu-backdrop"></div>' +

```

```

'<aside id="cap-menu-panel">' +
  '<div class="cap-menu-top">' +
    '<span class="cap-menu-tag">MENU</span>' +
    '<button id="cap-menu-close">CLOSE</button>' +
  '</div>' +
  '<nav class="cap-menu-links">' +
    '<div class="cap-menu-mask"><a data-target="900">About Us</a></div>' +
    '<div class="cap-menu-mask"><a data-target="3900">Portfolio</a></div>' +
    '<div class="cap-menu-mask"><a data-target="6200">Team</a></div>' +
    '<div class="cap-menu-mask"><a data-target="6942">Contact Us</a></div>' +
  '</nav>' +
  '<div class="cap-menu-row">' +
    '<span class="cap-menu-label">EXPLORE</span>' +
    '<div class="cap-menu-info">' +
      '<a data-target="2140">The Cappella Edge</a><br>' +
      '<a data-target="2940">Our Journey</a><br>' +
      '<a data-target="5200">Operators</a>' +
    '</div>' +
  '</div>' +
  '<div class="cap-menu-row">' +
    '<span class="cap-menu-label">STAY IN TOUCH</span>' +
    '<div class="cap-menu-info">' +
      '<a href="mailto:partner@cappella.in">partner@cappella.in</a><br>' +
      'Suite 303, Niharika Jubilee One, Jubilee Hills,<br>Road No. 1, Hyderabad, Telangana 500033' +
    '</div>' +
  '</div>' +
  '<div class="cap-menu-email">' +
    '<input type="email" placeholder="Email address">' +
    '<button aria-label="Subscribe">#8594;</button>' +
  '</div>' +
'</aside>';
document.body.appendChild(overlay);

// Staggered rise for the big links (transition-delay set per index)
const links = [...overlay.querySelectorAll('.cap-menu-links a')];
links.forEach((a, i) => { a.style.transitionDelay = (0.12 + i * 0.07) + 's, 0s'; });

// Dynamic scroll-based visibility check
const updateMenuBtnVisibility = () => {
  if (overlay.classList.contains('cap-menu-open')) return;
  const vp = document.getElementById('cap-viewport');
  const s = vp ? vp.clientWidth / 1440 : 1;
  const threshold = 180 * s;
  if (window.scrollY > threshold) {
    btn.style.opacity = '1';
    btn.style.pointerEvents = 'auto';
  } else {
    btn.style.opacity = '0';
    btn.style.pointerEvents = 'none';
  }
};

const open = () => {
  overlay.classList.add('cap-menu-open');
  btn.style.opacity = '0';
  btn.style.pointerEvents = 'none';
  if (this._lenis) this._lenis.stop();
  document.body.style.overflow = 'hidden';
};

const close = () => {
  overlay.classList.remove('cap-menu-open');
  updateMenuBtnVisibility();
  if (this._lenis) this._lenis.start();
  document.body.style.overflow = '';
};

btn.addEventListener('click', open);
overlay.querySelector('#cap-menu-close').addEventListener('click', close);
overlay.querySelector('#cap-menu-backdrop').addEventListener('click', close);
this._onMenuKey = (e) => { if (e.key === 'Escape') close(); };
window.addEventListener('keydown', this._onMenuKey);

```



```

// Listen to scroll to update button visibility
window.addEventListener('scroll', updateMenuBtnVisibility, { passive: true });
this._updateMenuBtnVisibility = updateMenuBtnVisibility; // Store reference for clean-up
updateMenuBtnVisibility(); // Run initially

// Section links: close, then glide to the target (frame px -> page px)
[...overlay.querySelectorAll('a[data-target]')].forEach(a => {
  a.addEventListener('click', () => {
    const frameTop = parseFloat(a.dataset.target);
    const vp = document.getElementById('cap-viewport');
    const target = frameTop * ((vp ? vp.clientWidth : window.innerWidth) / 1440);
    close();
    setTimeout(() => {
      if (this._lenis) this._lenis.scrollTo(target);
      else window.scrollTo({ top: target, behavior: 'smooth' });
    }, 250);
  });
});

// Email subscribe - same behaviour as the footer input
const email = overlay.querySelector('.cap-menu-email input');
overlay.querySelector('.cap-menu-email button').addEventListener('click', () => {
  if (email.value && email.value.includes('@')) {
    alert('Thank you for subscribing, ' + email.value + '!');
    email.value = '';
  } else {
    alert('Please enter a valid email address.');
```

18 — Film grain

CSS · lines 158-180

_setupGrain() · lines 1882-1887

Where: Fixed overlay across the entire page (z-index 95).

How: An inline-SVG feTurbulence noise tile at 5% opacity with mix-blend-mode: overlay, jittered by a 7-step steps() keyframe loop so the noise boils like film grain.

CSS — grain element + keyframes (lines 158-180)

```
/* Film grain - subtle animated noise over everything */
#cap-grain {
  position: fixed;
  inset: -120px;
  z-index: 95;
  pointer-events: none;
  opacity: 0.05;
  mix-blend-mode: overlay;
  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='280'
height='280'%3E%3Cfilter id='n'%3E%3CfeTurbulence type='fractalNoise' baseFrequency='0.85' numOctaves='2'
stitchTiles='stitch'/%3E%3CfeColorMatrix type='saturate' values='0'/%3E%3C/filter%3E%3Crect width='100%25'
height='100%25' filter='url(%23n)'/%3E%3C/svg%3E");
}
@media (prefers-reduced-motion: no-preference) {
  #cap-grain { animation: capGrain 0.7s steps(7) infinite; }
}
@keyframes capGrain {
  0% { transform: translate(0, 0); }
  14% { transform: translate(-38px, 22px); }
  28% { transform: translate(24px, -44px); }
  42% { transform: translate(-52px, -18px); }
  56% { transform: translate(40px, 36px); }
  70% { transform: translate(-20px, 48px); }
  84% { transform: translate(46px, -12px); }
  100% { transform: translate(0, 0); }
}
```

Full function — _setupGrain() (lines 1882-1887)

```
_setupGrain() {
  if (document.getElementById('cap-grain')) return;
  const g = document.createElement('div');
  g.id = 'cap-grain';
  document.body.appendChild(g);
}
```

19 — Inactive systems

`_setupMetaLabels()` · lines 1889–1918

`_setupJourneyDISABLED()` · lines 2261–2279

`meta-label CSS` · lines 182–194

Where: Meta labels: editorial [01 — THE CAPPELLA EDGE] tags per section (call commented out at line 706). JourneyDISABLED: an older scroll-linked journey phase crossfade superseded by the phase slider.

How: Included for completeness — both are fully functional but not invoked.

CSS — `.cap-meta` (lines 182–194)

```
/* Editorial section meta-labels: [ 01 — THE CAPPELLA EDGE ] */
.cap-meta {
  position: absolute;
  font-family: 'Space Mono', monospace;
  font-size: 13px;
  letter-spacing: 2px;
  white-space: nowrap;
  z-index: 5;
  opacity: 0;
  transform: translateY(14px);
  transition: opacity 0.8s var(--cap-ease-expo), transform 0.8s var(--cap-ease-expo);
}
.cap-meta.cap-meta-in { opacity: 0.85; transform: translateY(0); }
```

Full function — `_setupMetaLabels()` (lines 1889–1918)

```
_setupMetaLabels() {
  const scaler = document.getElementById('cap-scaler');
  if (!scaler || scaler.querySelector('.cap-meta')) return;
  // Anchored just above each section heading (frame coordinates)
  const defs = [
    ['01', 'THE CAPPELLA EDGE', 821, 2158, 'rgb(209,32,47)'],
    ['02', 'OUR JOURNEY', 45, 2996, 'rgb(255,255,255)'],
    ['03', 'OUR PORTFOLIO', 210, 4171, 'rgb(209,32,47)'],
    ['04', 'OPERATORS', 775, 5234, 'rgb(209,32,47)'],
    ['05', 'CONTACT', 44, 6990, 'rgb(255,255,255)']
  ];
  const io = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        entry.target.classList.add('cap-meta-in');
        io.unobserve(entry.target);
      }
    });
  }, { threshold: 0.5 });
  defs.forEach(([num, text, left, top, color]) => {
    const el = document.createElement('div');
    el.className = 'cap-meta';
    el.textContent = `[ ' + num + ' — ' + text + ' ]`;
    el.style.left = left + 'px';
    el.style.top = top + 'px';
    el.style.color = color;
    scaler.appendChild(el);
    io.observe(el);
  });
}
```

Full function — `_setupJourneyDISABLED()` (lines 2261–2279)

```
_setupJourneyDISABLED() {
  const j = document.getElementById('cap-journey');
```

```
if (!j) return;
const phases = [...j.querySelectorAll('.cap-phase')];
const clamp = (v) => v < 0 ? 0 : v > 1 ? 1 : v;
this._onJourney = () => {
  const sc = document.getElementById('cap-scaler');
  const m = (sc.style.transform || '').match(/scale\(((^)+)\)/);
  const s = m ? parseFloat(m[1]) : 1;
  const topPage = 2959 * s, hPage = 753 * s;
  const vh = window.innerHeight;
  const prog = clamp((window.scrollY + vh - topPage) / (hPage + vh));
  const idx = prog < 0.38 ? 0 : prog < 0.68 ? 1 : 2;
  phases.forEach((p, i) => { p.style.opacity = i === idx ? '1' : '0'; });
};
this._onJourney();
window.addEventListener('scroll', this._onJourney, { passive: true });
window.addEventListener('resize', this._onJourney);
}
```

20 — Animation wiring + cleanup

componentDidMount wiring · lines 690–716

componentWillUnmount · lines 3242–3265

Where: How everything above gets booted and torn down.

How: All systems are invoked from componentDidMount (some deferred by timeouts so the Figma frame settles first); componentWillUnmount removes every listener, interval, and rAF loop.

Boot sequence (excerpt of componentDidMount) (lines 690–716)

```
this._setupDotGrid();
this._setupNavScramble();

// ===== PREMIUM PASS: film grain (viewport-level, no frame needed) =====
// OCI reference: native cursor only – no custom cursor, ring, or spotlight
this._setupGrain();
this._setupMenuOverlay();

// ===== DEFERRED ANIMATIONS =====
// Wait for the Figma frame (x-import) to hydrate its DOM before querying elements.
// The frame renders async, so 1.6s covers the worst case render + font settle.
this._t3 = setTimeout(() => {
  this._setupScrollReveals();
  this._setupStatCounters();
  this._setupPortfolioInteraction();
  this._setupAccordion();
  // this._setupMetaLabels();
  this._setupJourneyStory();
  this._setupFooterAnimations();
  this._setupOperatorsMarquee();
  this._setupDitherEntries();
}, 1600);

// Hero intro: slight delay so first-frame paint is visible before elements vanish
this._t4 = setTimeout(() => this._setupHeroIntro(), 200);
this._t5 = setTimeout(() => this._setupHeroTitle(), 250);
}
```

Full function — componentWillUnmount() (lines 3242–3265)

```
componentWillUnmount() {
  window.removeEventListener('resize', this._fit);
  clearTimeout(this._t1); clearTimeout(this._t2);
  clearTimeout(this._t3); clearTimeout(this._t4); clearTimeout(this._t5);
  if (this._onMenuKey) window.removeEventListener('keydown', this._onMenuKey);
  if (this._clockIv) clearInterval(this._clockIv);
  if (this._mqRaf) cancelAnimationFrame(this._mqRaf);
  if (this._onRevealScroll) {
    window.removeEventListener('scroll', this._onRevealScroll);
    window.removeEventListener('resize', this._onRevealScroll);
  }
  if (this._onPortfolioScroll) {
    window.removeEventListener('scroll', this._onPortfolioScroll);
  }
  if (this._alignLogo) {
    window.removeEventListener('resize', this._alignLogo);
  }
  if (this._updateMenuBtnVisibility) {
    window.removeEventListener('scroll', this._updateMenuBtnVisibility);
  }
  if (this._scrollRevealIO) this._scrollRevealIO.disconnect();
}
```

```
renderVals() { return {}; }  
}
```